ChatCatHat^{TM}: Privacy Calling Someone

Submitted by

Liu Zhihang

School of Information and Software Engineering
University of Electronic Science and Technology of China

The final year project work was carried out under the 3+1+1 Educational Framework at the National University of Singapore (Chongqing) Research Institute

May 2026

ABSTRACT

Today, phone numbers are deeply linked to personal privacy. Traditional phone calls easily expose user data, leading to privacy leaks and network tracking. Existing solutions have obvious flaws: pure internet calling apps (VoIP) require both users to install the same software, while traditional "privacy numbers" are often blocked as spam by modern smartphones.

To solve these issues, this project designs and implements ChatCatHat™, a privacy-preserving calling system that combines the advantages of VoIP and traditional phone networks (PSTN). We introduce a secure proxy gateway between the caller and the receiver. The caller makes the call via an encrypted internet connection. The cloud server then hides their real identity, assigns a new number, and initiates a standard phone call to the receiver.

The caller connects via an encrypted VoIP link, and after the cloud server completes identity anonymization, number scheduling, and signaling rewriting, the system drives a media gateway to initiate a standard PSTN call to the target. This hybrid architecture achieves "clientless" one-way compatibility on the callee side, completely blocking the direct mapping of real identities between both parties, and effectively defending against metadata-based targeted tracking. This thesis details the architectural design and core component implementation of the middleware, proposes Quality of Service (QoS) optimization strategies for weak network environments, and comprehensively evaluates the system's communication performance and privacy defense capabilities.

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Professor Hari Grag Krishna, for his invaluable guidance, continuous support, and immense patience throughout the course of this Final Year Project. His profound academic insights and expert advice have been instrumental in shaping the direction of this research. Whenever we encountered bottlenecks in the architectural design or technical implementations of the privacy calling system, his encouragement and constructive feedback always guided us toward the right solutions.

I am also deeply grateful to my project partner, Chen Youhao. The successful implementation of the ChatCatHat™ system would not have been possible without our seamless collaboration. From the initial conceptualization to the countless hours of debugging network protocols and optimizing the system's performance, his dedication, technical skills, and reliable support have been indispensable. It has been a great privilege to work alongside such a dedicated partner.

Finally, I would like to extend my sincere thanks to the National University of Singapore (Chongqing) Research Institute and my home university for providing the excellent academic platform and resources that made this research endeavor possible.

CONTENTS

ABSTRACT **i**

ACKNOWLEDGEMENTS **ii**

LIST OF TABLES **v**

LIST OF FIGURES **vii**

LIST OF SYMBOLS **viii**

1 INTRODUCTION **1**

1.1 Background 1

1.1.1 Security of Digital ID via PSTN Number 1

1.1.2 Security Issues of Metadata Leakage 2

1.2 Problem Statement and Objective 3

1.3 Motivation 5

1.4 Main Contributions 6

1.5 Thesis Organization 7

2 Market Analysis **8**

2.1 Global Sec-Comm Market Analysis 8

2.1.1 Overall Development Trends 8

2.1.2 Niche Vertical: E2EE amd EaaS 9

2.2 Major Privacy Breach Case Study 12

2.2.1 Centralized Data Theft: PRISM 12

2.2.2 Parasitic Data Harvesting: Cambridge Analytica 15

2.2.3 Supply Chain Attack: SUNBURST 18

2.3 Global Regulatory and Compliance Analysis 21

2.3.1 EU: GDPR and NIS2 21

2.3.2 United States: STIR/SHAKEN 23

2.3.3 PRC: CSL,DSL and PIPL 24

2.3.4 ChatCatHat^{TM}: Far More Than a Simple Tool 26

3 Competitive Analysis and Requirement Analysis **28**
3.1 Competitive Analysis 28
3.1.1 Category I: Pure VoIP Platforms 28
3.1.2 Category II: Pure PSTN Virtual-Number Services 31
3.1.3 Category III: AXB Relay Platforms (O2O Virtual Number Intermediaries) 33
3.1.4 Cross-Category Competitive Positioning of ChatCatHat 35
3.2 Requirements Analysis 37
3.2.1 Functional Requirements 38
3.2.2 Non-Functional Requirements 42
3.2.3 Security Requirements 44
3.2.4 Privacy Requirements 46

4 System Design **50**
4.1 Privacy Calling Architecture Design 51
4.1.1 Dual Callback Chain 51
4.1.2 Number Selection Strategies 53
4.1.3 Control Plane Finite State Machine Design 55
4.2 System Front-end Design 57
4.2.1 Front-end UI Layout Design 57
4.2.2 Front-end UI Logic Design 59
4.3 Server Back-end Design 61
4.3.1 Back-end Function Design 61
4.3.2 Database Design 62
4.3.3 System Deployment 63

5 RESULTS AND EVALUATION **65**

6 CONCLUSIONS AND FUTURE WORK **66**
6.1 Conclusions 66
6.2 Future Work 66

REFERENCES **67**

A APPENDIX TITLE **68**

LIST OF TABLES

3.1 Pure VoIP Platforms: Comparative Summary 31
3.2 PSTN Virtual-Number Services: Comparative Summary 33
3.3 AXB Relay Platforms: Comparative Summary 35
3.4 Grand Competitive Comparison: ChatCatHat vs. All Categories . . . 37
3.5 Functional Requirements: VoIP Client 39
3.6 Functional Requirements: Middleware Core 40
3.7 Functional Requirements: Number Pool Management 41
3.8 Functional Requirements: User Account 42
3.9 Non-Functional Requirements 44
3.10 Security Requirements 46
3.11 Privacy Requirements 48

4.1 Three-Layer Decoupled Architecture of ChatCatHat™ 50
4.2 Identity Selection Policy Φ Scheduling Scenarios 55

LIST OF FIGURES

1.1 Digital Identity Hub 1
1.2 PRISM Tasking Process 3
1.3 Privacy-Preserving Telephony Architecture via Information-Asymmetric Middleware 5

2.1 Global Sec-Comm Market(In USD Bn) 9
2.2 E2EE Communication Market Expansion 10
2.3 EaaS Market Expansion Prediction till 2033 11
2.4 PRISM Data Collection Technical Architecture 13
2.5 How NSA steals data throughout PRISM 15
2.6 Cambridge Analytica 16
2.7 Cambridge Analytica Info Theft via Facebook Graph API 17
2.8 SUNBURST Highjack Timeline 19
2.9 SUNBURST Supply Chain Attack Architecture 20
2.10 EU NIS2 Act 23
2.11 The U.S. STIR/SHAKEN Standard Architecture 25
2.12 China's "Three Pillars" Data Protection Legislative Framework . . . 26
2.13 Regulatory Convergence and ChatCatHat™ Compliance-by-Design Positioning 27

3.1 Signal Messaging Architecture 29
3.2 WhatsApp 30
3.3 Telegram 30
3.4 Google Voice 32
3.5 Cross-Category Competitive Positioning of ChatCatHat™ 36
3.6 Functional Decomposition of ChatCatHat™ 43
3.7 Security Requirements: Defense-in-Depth Layer Model 47
3.8 Information-Asymmetric Architecture: Subsystem Data Isolation . . . 49

4.1 Privacy Calling Server Architecture 51
4.2 API-initiated call flow sequence diagram 52
4.3 Identity Selection Policy Φ Flow Chart 54
4.4 Professional State Machine in Life Cycle of 1 Single Call 55

4.5 Server Front-end Snapshot 1 58
4.6 Server Front-end Snapshot 2 58
4.7 Server Front-end Snapshot 3 59
4.8 Server Front-end Logic Design 60
4.9 Backend Detailed Architecture 61
4.10 Database Entity-Relationship Diagram 63
4.11 System Deployment Flow 64

LIST OF SYMBOLS AND ABBREVIATIONS

F1	Formula 1
ML	Machine Learning
API	Application Programming Interface

Chapter 1

INTRODUCTION

1.1 Background

1.1.1 Security of Digital ID via PSTN Number

Amidst the rapid evolution of communication networks, the fundamental role of phone numbers has undergone a paradigm shift. In the traditional Public Switched Telephone Network (PSTN), E.164-based numbers served merely as routing endpoints for circuit switching. Today, driven by the ubiquitous mobile internet and real-name registration mandates, phone numbers have morphed into ultimate "digital identity anchors." They are no longer just communication identifiers but are intricately bound to users' social networks, financial accounts, and public services (as shown in Figure 1). The prohibitive switching costs—stemming from the tedious process of unbinding accounts, updating financial details, and notifying contacts—compel most users to retain a single, static number for extended periods.However, this chronically exposed and static identifier significantly amplifies the risk of privacy breaches. Once compromised in a single service node, a phone number becomes a prime vector for cross-platform correlation attacks by malicious actors.

Pic/1.1.png

Figure 1.1: Digital Identity Hub

1.1.2 Security Issues of Metadata Leakage

The exposure of communication metadata further amplifies the security risks associated with identity binding. Although VoIP technology has significantly reduced routing costs, this benefit comes at the expense of exposing communication signaling directly to the complexities of the IP network environment. Taking the widely adopted SIP protocol as an example: even if the RTP media streams carrying voice traffic are end-to-end encrypted, their signaling headers continue to transmit core metadata—such as caller identity, destination IP, timestamps, and call duration—in plaintext. Attackers need not crack the actual content of the conversation; the mere collection and correlational analysis of this plaintext metadata suffice to accurately reconstruct a target user's social graph and behavioral patterns.

These vulnerabilities—rooted in the exposure of phone numbers and associated metadata—have been starkly validated by numerous cybersecurity and surveillance scandals that have shocked the world in recent years. The most emblematic instance of such "downstream surveillance" is the "PRISM" scandal, exposed in 2013; in this case, surveillance agencies bypassed the high costs associated with physical wiretapping, opting instead to bulk-extract user communication records and other metadata from internet giants via compliant interfaces or system backdoors. As it turned out, even without the ability to decrypt the actual content of end-to-end encrypted communications, the mere long-term collection and collation of metadata proved sufficient to precisely reconstruct the social graphs and behavioral patterns of target populations.

A far more severe form of targeted threat stems from advanced, military-grade spyware such as Pegasus. Attackers often require nothing more than the target's phone number to exploit underlying vulnerabilities within applications like WhatsApp or iMessage; by silently delivering "zero-click" malicious payloads through communication channels, they can seize full administrative control over a mobile device without requiring any interaction whatsoever from the target user.

Pic/1.2.jpg

Figure 1.2: PRISM Tasking Process

1.2 Problem Statement and Objective

Addressing the various shortcomings of traditional telephony systems—specifically regarding privacy protection and system robustness—as outlined above, this privacy-preserving telephony system aims to establish an information-asymmetric communication architecture through the introduction of software-defined communication middleware. Its objective is to conceal the true identity of the call initiator while ensuring that call connectivity remains fully unimpeded. To guarantee the system's high practical feasibility and ensure a seamless deployment process, strict boundary constraints were established during its engineering implementation: all system modifications and middleware interventions are strictly confined to the call initiator's (Caller) side, requiring absolutely no modifications to the underlying carrier's VoIP core network infrastructure or the recipient's terminal device.

To rigorously describe the implementation mechanism of private calls, this system employs set theory and mapping functions to mathematically model the underlying communication processes.

First, we define the set of all possible caller identities (Caller IDs) as an infinite set $\mathbb{I}$. Within this space, let the real identity of the caller be $id_C \in \mathbb{I}$, and the real

identity of the callee be $id_T \in \mathbb{I}$. The system concurrently maintains a finite set of "virtual identities" $\mathbb{V} \subset \mathbb{I}$, which serves as the system's available number pool, satisfying the condition $id_C \notin \mathbb{V}$.

We define the public telephone network operator as an operator $\mathcal{N}$, which receives an ordered pair of identities (id_{from}, id_{to}) and establishes the underlying physical connection. The perception function of the callee's terminal is denoted as $Perceive_T(\cdot)$, which is used to parse the information transmitted by the operator and return the call source. In a conventional call, the network operator directly executes $\mathcal{N}(id_C, id_T)$, and the callee's perception result is $Perceive_T(\mathcal{N}(id_C, id_T)) = id_C$, resulting in the exposure of the real identity.

To sever the transmission of the real identity, the core of the privacy middleware is an identity transformation function Φ. When the caller initiates a call intent $Request_{init}(C, M, id_T)$, the middleware captures this request and applies the identity transformation function:

$$\Phi : (\mathbb{I} \times \mathbb{I}) \to (\mathbb{V} \times \mathbb{I})$$

$$\Phi(id_C, id_T) = (id_v, id_T), \quad \text{where } id_v \leftarrow select(\mathbb{V}, id_C, id_T)$$

Here, $select(\cdot)$ represents the number selection strategy (such as a callee consistency strategy). Subsequently, the middleware submits the transformed masqueraded call pair to the network operator to execute $\mathcal{N}(id_v, id_T)$. The perception result on the callee side becomes $Perceive_T(\mathcal{N}(id_v, id_T)) = id_v \neq id_C$, logically closing the proof of privacy protection in a mathematical sense.

The critical constraints of this project at the requirements level are explicitly defined: to minimize modifications to the operator's links, to eliminate the need for installing additional middleware on the callee's end, and to prioritize the system's deployability and verifiability. Under these constraints, the project opts to handle the call control logic via a middleware layer. By utilizing an identity mapping strategy, it transforms the "real caller identity" into a "manageable virtual identity," and enables both parties to complete the call through media bridging without perceiving the complex underlying control flow. This problem definition dictates that the system must simultaneously satisfy three categories of objectives. The first is the business objective: ensuring calls can be established and the caller's identity

can be substituted. The second is the engineering objective: ensuring states are traceable and faults can be localized. The third is the operational objective: ensuring services are hostable and operations are auditable. The project's current code implementation is precisely centered around realizing these three objectives.

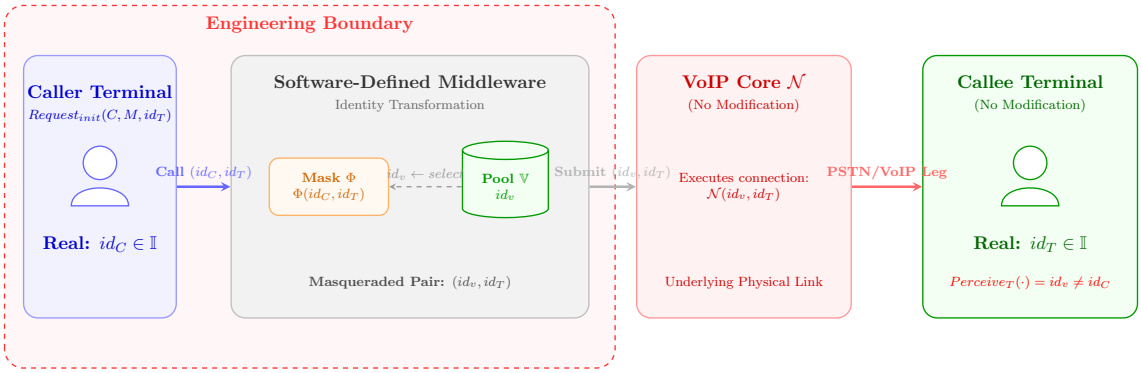

Figure 1.3: Privacy-Preserving Telephony Architecture via Information-Asymmetric Middleware

1.3 Motivation

In the face of escalating digital security threats, traditional mitigation strategies are proving increasingly inadequate. The industry's prevailing approach heavily relies on reactive patching of obsolete systems or shifting the burden and cost of privacy protection onto end-users. However, with the normalization of widespread data surveillance and the democratization of weaponized cyber threats, these superficial application-layer defenses—such as encrypting content while leaving core metadata fully exposed—are experiencing systemic failure. On a macro level, there is an urgent necessity for a paradigm shift from reactive patching to structural "Privacy by Design." Future communication systems must inherently decouple "identity entities" from "communication utility" at the foundational level, ensuring that the fundamental right to participate in the digital world does not come at the expense of personal privacy.

The ultimate motivation of this project is to reclaim the structural sovereignty of personal data within existing global communication networks. By introducing a software-defined decoupling layer without disrupting current infrastructure, this endeavor transcends the limitations of a mere standalone tool, striving instead to

establish a new standard for identity management and privacy protection in a decentralized era. We are committed to democratizing high-level privacy-enhancing technologies, making them universally accessible and capable of zero-friction deployment. Ultimately, this initiative is a crucial step toward building a more equitable and secure digital ecosystem—one where the ability to communicate freely and anonymously is no longer a premium privilege for the few, but an inalienable default right for every digital citizen.

1.4 Main Contributions

Through systematic architectural innovation and algorithmic optimization, this research provides a viable technological paradigm for digital identity privacy protection without disrupting existing telecommunications infrastructure. The core contributions are summarized as follows:

- **Zero-Modification Privacy-Preserving Architecture:** Proposed a decoupled three-tier architecture (Control Plane, Media Plane, and State Plane) to establish an information-asymmetric communication system. By utilizing a "Double-Callback" mechanism based on the Back-to-Back User Agent (B2BUA) model, the system effectively conceals the caller's real identity.

- **Callee-Consistent Identity Selection Policy:** Introduced a Callee-Consistent Identity Selection Policy. This algorithm establishes and maintains a long-term weak binding relationship between the callee's real number and a specific virtual number. By intelligently managing historical mappings, idle status, and fallback mechanisms during concurrent conflicts, the system significantly enhances call trust and connectivity rates without sacrificing privacy.

- **System Capacity and Concurrency Modeling:** Constructed a mathematical capacity model tailored for finite-source user groups using the Engset model. It accurately calculates the blocking probability and dynamic scaling requirements for the system.

- **Blockchain-Based Decentralized Authentication:** Integrated a blockchain and Web3-based authentication paradigm. It utilizes asymmetric cryptogra-

phy, dynamic Nonce challenges, and smart contract verification to replace static API keys. This prevents Man-in-the-Middle (MITM) attacks and single points of failure, establishing a secure, zero-trust boundary for node access.

1.5 Thesis Organization

Chapter 2

Market Analysis

2.1 Global Sec-Comm Market Analysis

2.1.1 Overall Development Trends

The global secure communications market is currently undergoing a phase of robust expansion, with its overall scale poised to experience a significant breakthrough over the coming decade. According to chart data from Wise Guy Reports, global revenue in the secure communications market stood at approximately $6.86 billion in 2024, subsequently climbing to $7.28 billion in 2025. Driven by the continuous deepening and refinement of global digital infrastructure, the market is projected to maintain a Compound Annual Growth Rate (CAGR) of 6.1% throughout the forecast period spanning from 2025 to 2035. Following this growth trajectory, the total size of the global secure communications market is expected to nearly double by 2035, reaching $13.20 billion. This sustained upward trend underscores a pivotal shift: in today's hyper-connected modern society, the establishment of robust secure communication architectures has officially evolved from a mere supplementary requirement for select enterprises into a foundational infrastructure essential for underpinning the operations of the global digital economy and government administration.

The core drivers behind the robust expansion of this market stem primarily from increasingly sophisticated cybersecurity threats, the evolution of cutting-edge technologies, and tightening global data compliance regulations. With the explosive growth of Internet of Things (IoT) devices, the widespread adoption of cloud computing, and the deepening development of digital finance, the scope and destructive potential of cyberattacks—such as data breaches and ransomware—have escalated significantly. This has directly stimulated an urgent, non-negotiable demand for advanced encrypted communication solutions across highly sensitive sectors, including defense and military, financial services, government agencies, and healthcare. Furthermore, rapid breakthroughs in quantum computing technology pose a substantial threat to traditional encryption algorithms, compelling the entire industry to accelerate its transition toward next-generation security defense technologies, such as

Post-Quantum Cryptography (PQC) and Quantum Key Distribution (QKD). Compounded by the comprehensive implementation of stringent data protection regulations—exemplified by the GDPR—global enterprises are compelled to continuously increase their investment in secure communication platforms and end-to-end encryption verification solutions; collectively, these factors inject powerful momentum into the sustained prosperity of the global secure communication market.

Pic/2.1.png

Figure 2.1: Global Sec-Comm Market(In USD Bn)

2.1.2 Niche Vertical: E2EE amd EaaS

The global encryption market is currently experiencing explosive growth, driven by the complementary interplay between underlying technology delivery models (EaaS) and application-layer platforms (E2EE communication). According to data from Grand View Research, the global "Encryption-as-a-Service (EaaS)" market is projected to skyrocket from approximately $1.8 billion in 2024 to $12.8 billion by 2033—a surge primarily driven by the emergence of more comprehensive "solutions." Concurrently, Intel Market Research forecasts that the "End-to-End Encryption (E2EE) Communication Market" will soar from $6.12 billion in 2024 to $24.19 billion by 2031, exhibiting a staggering Compound Annual Growth Rate of 21.7%.The high-growth trends exhibited by both, however, are remarkably consistent, reflecting that cryptographic technology has become an indispensable, fundamental necessity within the digital economy.

As an underlying technology delivery model, EaaS provides the critical encryp-

tion algorithms, key management capabilities, and cloud-based delivery infrastructure required to implement high-end E2EE communication platforms. In essence, EaaS solutions constitute the technical core of E2EE communication tools, and their growth serves as the technical foundation enabling the widespread adoption of the E2EE communication market. Regarding specific market catalysts, the escalating threat of cybercrime—exemplified by the "Salt Typhoon" hacking campaign targeting U.S. government officials' communications in late 2024—directly demonstrates the inherent vulnerabilities of traditional non-encrypted or weakly encrypted communication methods. Consequently, this has compelled governments and critical sectors to mandate an immediate and comprehensive transition to E2EE communication tools, thereby directly driving "state-level" procurement demand within the E2EE communication market.

Pic/2.2.png

Figure 2.2: E2EE Communication Market Expansion

Furthermore, technological iteration and regulatory pressure constitute a dual engine driving market growth. With the looming threat of quantum computing, the U.S. National Institute of Standards and Technology (NIST) has officially released Post-Quantum Cryptography (PQC) standards, compelling governments and critical infrastructure enterprises worldwide to initiate a new wave of fundamental cryptographic technology overhauls. This technological iteration has not only triggered a surge in demand for upgrading End-to-End Encrypted (E2EE) communication systems but has also driven the procurement of more sophisticated and costly PQC-as-a-Service (PQC EaaS) solutions. Concurrently, the tightening of

global regulatory policies and the establishment of privacy legislation have—from the dual perspectives of compliance and fundamental human rights—acted as a powerful catalyst for market expansion. The EU's NIS2 Directive, which came into full effect in 2024, utilizes stringent regulations to mandate cryptographic technology as an essential requirement for cybersecurity risk management within critical entities. On the judicial front, the European Court of Human Rights (ECHR) delivered a landmark ruling declaring that "weakening encryption technologies—such as by mandating backdoors—constitutes a violation of fundamental human rights." This decision legally enshrined the inviolable status of E2EE, providing political and legal safeguards for the long-term, robust growth of both markets. Driven by the triple forces of mandatory compliance, heightened security thresholds, and the iteration of post-quantum technologies, End-to-End Encryption and Encryption-as-a-Service have decisively moved beyond their nascent stages to officially become indispensable, fundamental necessities within the global digital economy.

Pic/2.3.png

Figure 2.3: EaaS Market Expansion Prediction till 2033

2.2 Major Privacy Breach Case Study

2.2.1 Centralized Data Theft: PRISM

Technical Implementation

PRISM (internal codename US-984XN) is a large-scale internet communications data collection program operated by the U.S. National Security Agency (NSA) since 2007, authorized under the *Protect America Act* and Section 702 of the *FISA Amendments Act of 2008*. Unlike traditional targeted surveillance, PRISM's core architecture relies on a passive, centralized data-siphoning mechanism leveraged by judicially sanctioned legal authority: acting on behalf of the NSA, the FBI issues "selectors"—such as email addresses or instant messaging identifiers—to U.S. internet service providers bound by legal directives (specifically, Section 702 Directives). Under legal obligation, these companies must continuously hand over all communications matching these selectors until the government revokes the surveillance directive for a specific selector. Once initiated, this collection process remains in a state of perpetual activation, requiring no subsequent judicial authorization for each individual instance of data acquisition.

At the technical implementation level, NSA analysts input selectors via the Unified Targeting Tool (UTT); these selectors are restricted to strong identifiers—such as email addresses or IP addresses—rather than natural-language keywords. As disclosed by the French newspaper *Le Monde*, approximately 45,000 active selectors operate concurrently within the PRISM system. Analysts can retrieve data from two dimensions: first, Surveillance—the forward capture of a target's real-time communications from the moment a selector is activated; and second, Stored Comms—the retrospective retrieval of historical communication records previously archived by various service providers prior to the selector's activation. The types of data collected are extremely broad, encompassing audio and video chats, emails, documents, photos, VoIP call logs (e.g., Skype), file transfer logs, and social networking metadata. The technology companies participating in this data handover include Microsoft, Yahoo, Google, Facebook, YouTube, Skype, AOL, and Apple; according to an internal NSA memo leaked to *The Washington Post*, "98% of PRISM's intelli-

gence output is based on Yahoo, Google, and Microsoft." Beyond PRISM, the NSA also utilizes the complementary Upstream collection program—specifically through sub-programs such as FAIRVIEW, BLARNEY, and STORMBREW—to perform full-volume mirroring and Deep Packet Inspection (DPI) on internet traffic passing through physical aggregation nodes within the U.S. domestic fiber-optic backbone network, thereby achieving dual coverage of international communication pipelines. These two programs provide joint query capabilities via the REPRISM/FISA and XKEYSCORE indexing and analysis systems, funneling collected data into a unified, searchable intelligence database that enables analysts from the NSA, FBI, and CIA to conduct back-door searches—queries that include identifiers belonging to U.S. citizens.

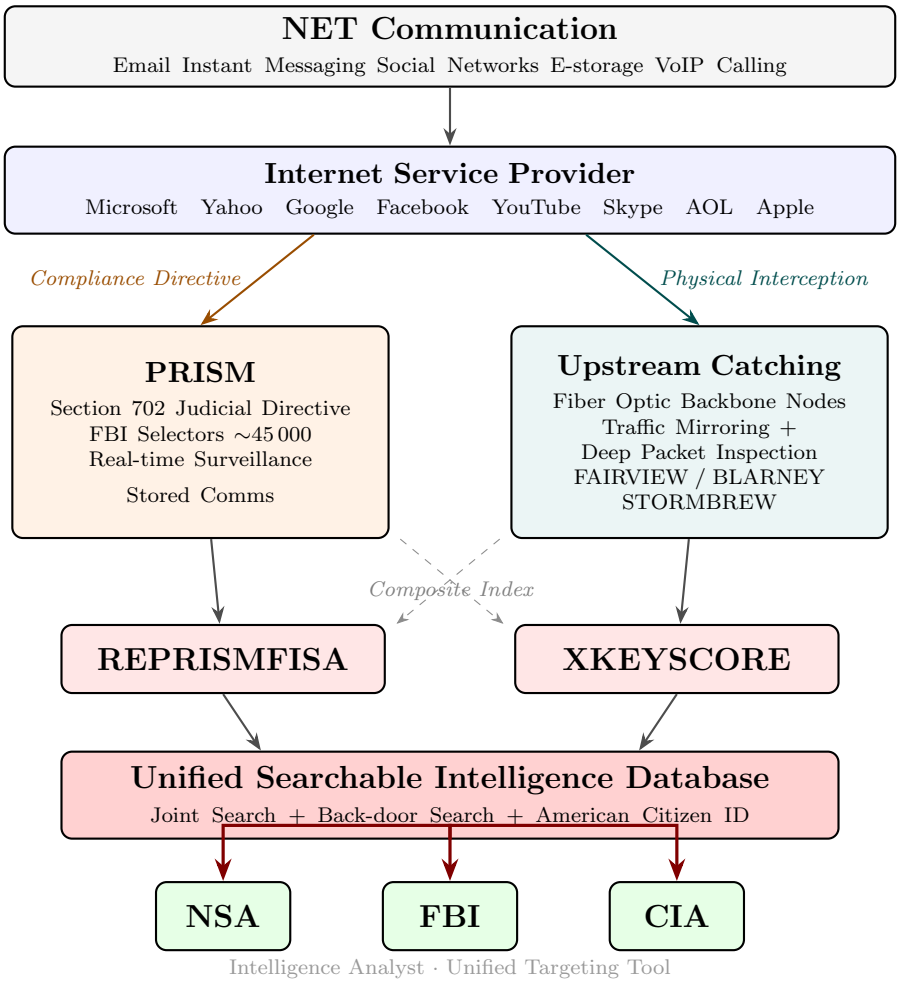

Figure 2.4: PRISM Data Collection Technical Architecture

Financial Loss and Society Impact

On June 6, 2013, former NSA contractor Edward Snowden leaked classified documents—including a 41-page PowerPoint training presentation—to *The Guardian* and *The Washington Post*, thereby exposing the existence of the PRISM program to the world. This event triggered an unprecedented global debate regarding the legitimate boundaries of state surveillance powers; its shockwaves extended far beyond the realm of intelligence and security, profoundly shaking the very foundations of trust within the global digital economy.

On the legal and political fronts, on September 2, 2020, a U.S. federal appeals court ruled that the NSA's mass metadata collection program was unlawful—and potentially unconstitutional—while noting that U.S. intelligence officials who had previously defended the program did not tell the truth. Laura Donohue, a law professor at Georgetown University, explicitly characterized PRISM and other NSA mass surveillance programs as unconstitutional. This event also directly catalyzed the legislative process for the European Union's General Data Protection Regulation (GDPR): German data protection authorities took the lead in calling for a suspension of all data transfers between the U.S. and Europe based on the Safe Harbor Agreement.

On the economic front, the PRISM scandal inflicted a severe trust deficit and actual market contraction upon the U.S. technology industry. A research report published in August 2013 by the Information Technology and Innovation Foundation estimated that the exposure of PRISM could cost the U.S. cloud computing industry between $21.5 billion and $35 billion over a three-year period. Forrester analyst James Staten offered an even more pessimistic forecast, suggesting that the net losses for the U.S. internet service provider sector could reach as high as $180 billion—equivalent to 25% of the total IT services market. The report noted that approximately 10% of non-U.S. respondents indicated they had already cancelled projects with U.S. cloud service providers due to PRISM, while 56% of respondents stated they were unlikely to use U.S. cloud services in the future. Artmotion, Switzerland's largest cloud hosting provider, recorded a 45% increase in revenue within just one month of Snowden's disclosures; meanwhile, various European nations accelerated the advancement of their data sovereignty strategies, moving to

establish localized European cloud infrastructure.

Pic/2.5.png

Figure 2.5: How NSA steals data throughout PRISM

2.2.2 Parasitic Data Harvesting: Cambridge Analytica

Technical Implementation

The technical roots of this incident can be traced back to the Open Graph platform launched by Facebook in 2010. This platform allowed third-party applications to request permission from users to access their personal data; **crucially**—it simultaneously empowered these applications to access the personal data of *all* of that user's Facebook friends. Early versions of Facebook Login (v1.0) employed an extreme "all-or-nothing" authorization model:when faced with an application's permission request, users could only choose to grant or deny access in its entirety, **without the ability to selectively decline specific permissions**. Furthermore, access to the friends list was categorized as "basic information"—on par with

a user's name and profile picture—a design classification that significantly increased the likelihood of users granting this permission.

In 2013, Aleksandr Kogan, a psychologist at the University of Cambridge, and his company, Global Science Research (GSR), developed a personality test application named thisisyourdigitallife. Operating under the guise of academic research, the application paid to recruit approximately 270,000 to 320,000 Facebook users to complete psychometric questionnaires. However, this marked merely the starting point of a data-harvesting operation: leveraging Facebook's Graph API, the application not only scraped the complete profile information of the test participants themselves but also—without obtaining secondary consent—recursively harvested the personal data of their entire networks of friends. This harvested data included user profiles, Like records, social connection graphs, geolocation data, and even portions of private messages. Through this mechanism of exponential diffusion across the social graph, the application ultimately harvested the profiles of up to 87 million Facebook users, all stemming from a base of only about 300,000 direct participants.

Pic/2.6.png

Figure 2.6: Cambridge Analytica

Financial Loss and Society Impact

Within a week of the scandal breaking, Facebook's stock price plummeted by over 13%, marking its third-worst weekly performance since going public in 2012. By March 26, 2018, Facebook's shares had fallen cumulatively by approximately 24%,

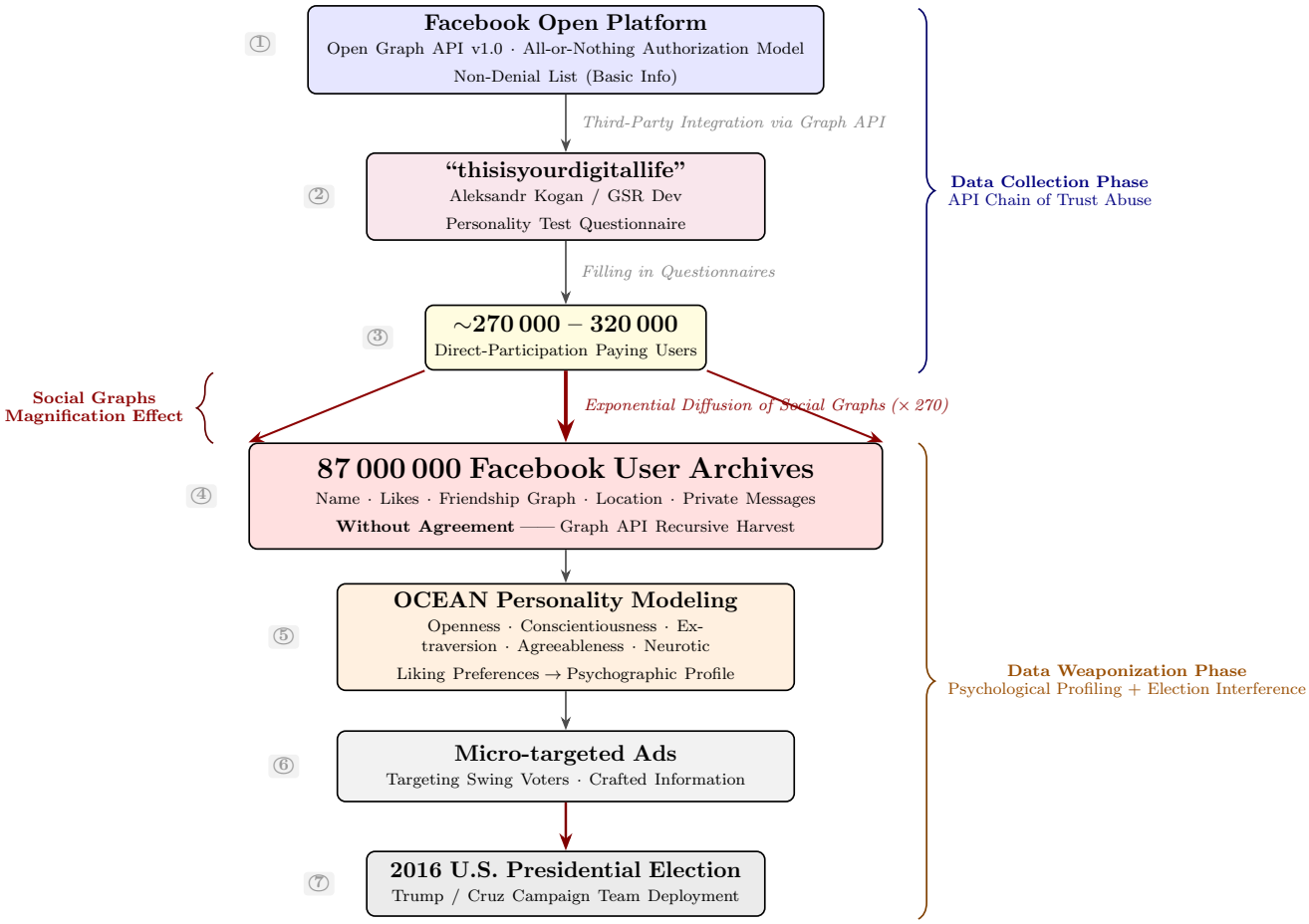

Figure 2.7: Cambridge Analytica Info Theft via Facebook Graph API

wiping out roughly $134 billion in market capitalization. The shockwaves were not confined to Facebook alone: during the same period, shares of Google's parent company, Alphabet, fell by about 7%, while Twitter plunged 20%, as the entire technology sector was hit by severe panic selling driven by regulatory fears.

In July 2019, the U.S. Federal Trade Commission (FTC) approved—by a 3-to-2 vote—a $5 billion fine against Facebook; this marked the largest penalty ever imposed by the FTC on a technology company. The UK's Information Commissioner's Office (ICO) fined Facebook £500,000 (the maximum amount permissible under the law at the time). Cambridge Analytica itself declared bankruptcy under the pressure of investigations spanning multiple countries. The governments of India and Brazil demanded that Cambridge Analytica disclose the specific methods it had employed to use data to interfere in political campaigns. On the level of public opinion, a global #DeleteFacebook movement gained momentum, with WhatsApp co-founder Brian Acton and Tesla CEO Elon Musk publicly joining the call.

2.2.3 Supply Chain Attack: SUNBURST

Technical Implementation

SolarWinds is a Texas-based vendor of IT infrastructure management software. Its flagship product, Orion, is a widely deployed network performance monitoring platform utilized by over 30,000 customers worldwide—including 499 of the U.S. Fortune 500 companies, as well as numerous U.S. federal government agencies. Attackers had infiltrated SolarWinds' internal network as early as September 2019, conducting extensive reconnaissance and familiarizing themselves with the environment during a dormant period lasting approximately five months. In October 2019, the attackers performed preliminary code injection tests within the Orion build environment. On February 20, 2020, the attackers formally deployed a highly customized malicious code injector dubbed SUNSPOT. This tool was embedded within SolarWinds' automated software build pipeline, specifically designed to inject backdoor code into a particular DLL component of the Orion platform—**SolarWinds.Orion.Core.BusinessLayer.dll**—during the compilation process, without ever touching the source code repository itself. Technical analysis by CrowdStrike indicates that SUNSPOT's design reflects a significant effort by its developers to ensure the code was injected correctly and remained undetected, prioritizing operational security to avoid exposing their presence within the build environment. Because the malicious code was injected during the build phase, the resulting compiled Orion update packages were signed with SolarWinds' official digital certificates, thereby appearing to customer systems—from a cryptographic standpoint—as completely legitimate and trusted software updates.

From March to June 2020, SolarWinds—without any knowledge of the compromise—pushed updates to its Orion platform containing the SUNBURST backdoor (affecting versions 2019.4 through 2020.2.1 HF1); approximately 18,000 customers downloaded and installed these infected updates. Upon activation, the SUNBURST backdoor first executes a series of rigorous anti-detection checks—verifying the target environment for the presence of antivirus software or forensic analysis tools—before establishing communication with Command and Control (C2) servers distributed across multiple domains, utilizing the HTTP protocol while masquerading as le-

gitimate Orion traffic. The attackers leased Virtual Private Servers (VPS) located within the United States, ensuring that the source IP addresses of the C2 traffic originated from the same country as the victims, thereby evading threat detection mechanisms based on geographic anomalies. In the subsequent exploitation phase, SUNBURST can deliver a memory-only payload dropper named TEARDROP, which further deploys a Cobalt Strike Beacon as a post-exploitation tool to achieve persistent access and lateral movement within the target network.

Pic/2.8.png

Figure 2.8: SUNBURST Highjack Timeline

Financial Loss and Society Impact

Regarding the scope of the impact, although approximately 18,000 customers downloaded the infected update, the attackers actually activated the backdoor—and subsequently engaged in deep exploitation—only against a select few high-value targets. According to Microsoft's analysis, approximately 40 organizations were confirmed to have suffered a substantive compromise; of these, 80% were located within the United States, 44% were information technology firms, and 18% were government agencies. The U.S. federal agencies targeted in the attack included core departments such as the Department of Defense (the Pentagon), the Department of Homeland Security, the Department of State, the Department of the Treasury, the Department of Energy, and the National Nuclear Security Administration. Affected private sector entities included global technology giants such as Microsoft, Cisco, Intel, and Deloitte. In the first quarter of 2021 alone, SolarWinds invested between $18 million and $19 million in incident investigation and remediation efforts (representing SolarWinds' own expenditures only).Experts estimate that the total cost of fully eradicating the malicious code and securing all affected systems could reach as high

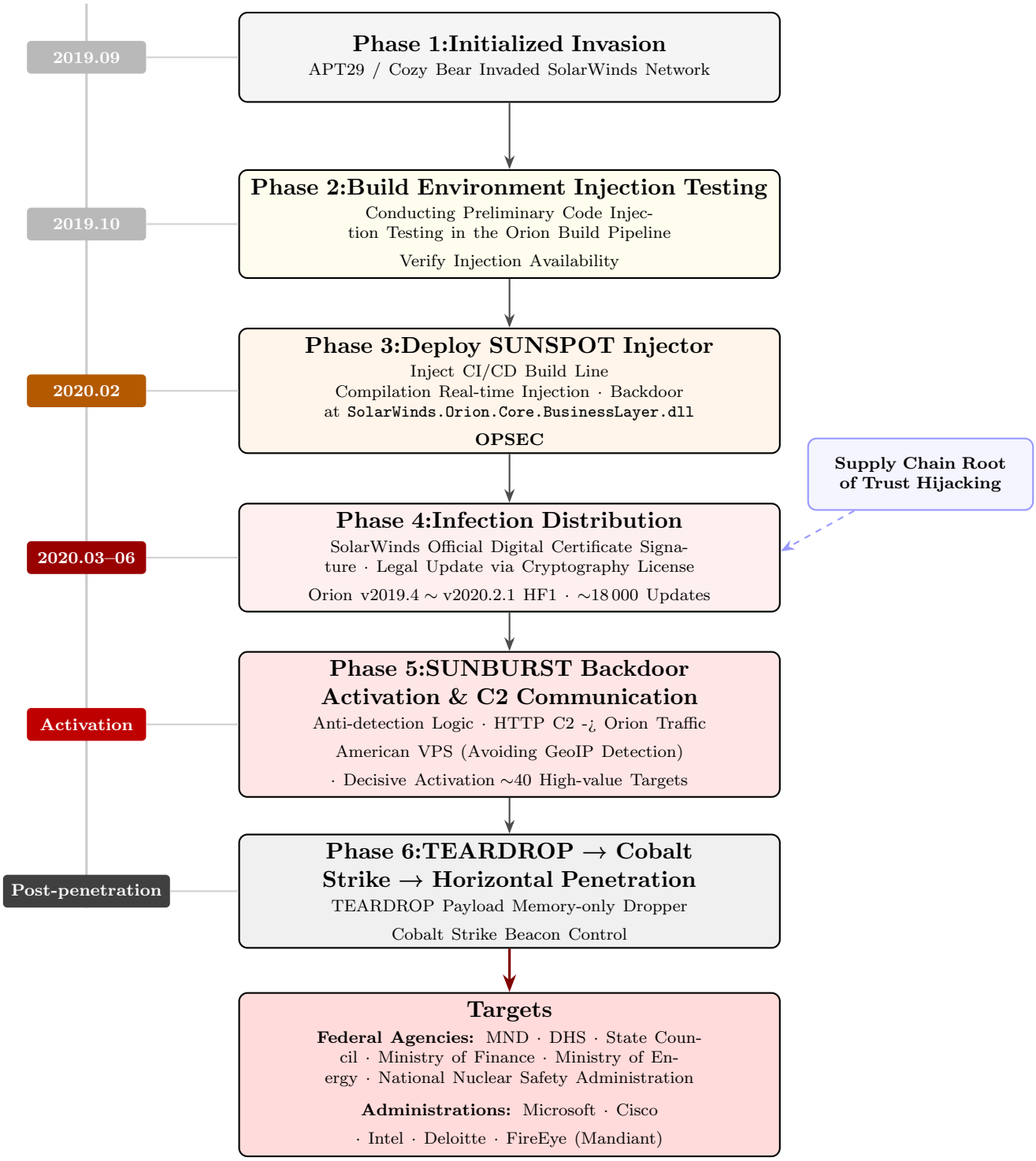

Figure 2.9: SUNBURST Supply Chain Attack Architecture

as $100 billion. An investigative report published by IronNet Cybersecurity in 2021 revealed that enterprises affected by the SolarWinds attack lost, on average, 11% of their annual revenue (with U.S. enterprises averaging a loss of 14%).

The strategic significance of this incident far exceeds its direct economic losses. For the first time—and on such a massive scale and with such profound depth—it

demonstrated to the world that:Within the highly interconnected ecosystem of the modern software supply chain, attackers can subvert a single trusted upstream vendor to achieve a "one-to-many" infiltration of thousands of downstream organizations; furthermore, traditional security frameworks—predicated on perimeter defenses and assumptions of local security—prove almost entirely ineffective when confronted with legitimately signed malicious updates.The SolarWinds incident directly precipitated the signing of Executive Order 14028, Improving the Nation's Cybersecurity, by U.S. President Biden in May 2021. This order mandates that federal agencies adopt Zero Trust Architecture, strengthen software supply chain security audits, and establish mechanisms such as the Software Bill of Materials (SBOM), thereby marking a fundamental paradigm shift in cybersecurity—moving away from traditional perimeter defenses toward supply chain trust verification.

2.3 Global Regulatory and Compliance Analysis

The aforementioned case studies have amply demonstrated how systemic vulnerabilities within communication infrastructure can be exploited on a massive scale. Consequently, governments worldwide and supranational bodies are progressively establishing multi-layered legal and regulatory frameworks designed to simultaneously elevate cybersecurity baselines, safeguard personal data, and—crucially for the purposes of this project—define the legitimate boundaries of communication privacy. This section will examine the primary regulatory regimes of the European Union, the United States, and the People's Republic of China, analyzing the specific implications these frameworks hold for the design and deployment of privacy-enhancing telephony middleware, such as ChatCatHat™.

2.3.1 EU: GDPR and NIS2

The European Union has established the most comprehensive and influential data protection and cybersecurity regulatory ecosystem in the world, anchored by three mutually reinforcing pillars: the General Data Protection Regulation (GDPR), the Network and Information Security Directive 2 (NIS2), and landmark jurisprudence from the European Court of Human Rights (ECtHR).

Since coming into effect in May 2018, the General Data Protection Regulation (GDPR) has explicitly identified encryption as a potential technical and organizational measure for safeguarding data security, as enumerated in Article 32, Paragraph 1. Although the Regulation intentionally avoids mandating specific encryption algorithms or methods—aiming instead to maintain technological neutrality and adapt to rapid technological advancements—its risk-based regulatory framework effectively provides a powerful incentive for the adoption of robust cryptographic protection measures. Data controllers must consistently assess the severity of the risks posed to the rights and freedoms of data subjects; the higher the risk—and the greater the likelihood of its occurrence—the more robust the required security measures must be. Furthermore, in the event of a data breach, regulatory authorities are required to actively consider whether the relevant parties had implemented encryption measures when determining whether to impose fines and the magnitude of such penalties. By the end of 2025, the cumulative total of fines levied under the GDPR had exceeded €6.7 billion—a fact that starkly underscores that the financial risks associated with non-compliance are not only substantial but also on a continuous upward trajectory.

Building upon the GDPR's data-centric protections, the NIS2 Directive (Directive (EU) 2022/2555) addresses cybersecurity at the network and systems level. The directive establishes a unified legal framework to uphold cybersecurity in 18 critical sectors across the EU and calls on Member States to define national cybersecurity strategies and collaborate for cross-border reaction and enforcement. Notably, Article 21 of NIS2 explicitly mandates that essential and important entities implement risk-management measures that include, at minimum, "policies and procedures regarding the use of cryptography and, where appropriate, encryption" as well as "the use of multi-factor authentication, continuous authentication solutions, voice, video, and text encryption, and encrypted internal emergency communication, when appropriate."The directive further stipulates that end-to-end encryption technology should be used by organizations "to safeguard the security of public electronic communications networks and publicly available electronic communications services." Non-compliance can result in fines of up to €10 million or 2% of global turnover, and—representing a significant shift—NIS2 requires corporate management to over-

see and approve cybersecurity measures, with breaches potentially resulting in personal liability and temporary bans from management roles.

Taken together, GDPR's data protection obligations, NIS2's cybersecurity mandatesas a fundamental right—create a regulatory environment in which privacy-preserving communication architectures are not merely permitted but are, in an increasing number of contexts, legally required.

Pic/2.10.jpg

Figure 2.10: EU NIS2 Act

2.3.2 United States: STIR/SHAKEN

In contrast to the EU's comprehensive rights-based approach, the United States has addressed communication privacy through a combination of sector-specific regulations and industry-driven technical standards, with a particular focus on combating caller ID spoofing—a problem directly relevant to the design of ChatCatHat™.

The STIR/SHAKEN framework is an industry-standard caller ID authentication technology, consisting of a set of technical standards and protocols that allow for the authentication and verification of caller ID information for calls carried over Internet Protocol (IP) networks.STIR (Secure Telephone Identity Revisited) has been defined as a series of RFC standards by the Internet Engineering Task Force (IETF), and works by adding a digital certificate to the Session Initiation Protocol (SIP) information used to initiate and route calls in VoIP systems. Under this framework, when a call is initiated, the originating service provider assesses the call's source and assigns one of three attestation levels: Full Attestation (A)—the

provider has authenticated the calling party and they are authorized to use the calling number; Partial Attestation (B)—the provider has authenticated the call origination but cannot verify the call source is authorized to use the calling number; and Gateway Attestation (C)—the provider has authenticated from where it received the call but cannot authenticate the call source.

However, the STIR/SHAKEN framework introduces specific technical challenges for privacy-preserving telephony systems. The system is based on SIP and is designed to work with calls being routed through a VoIP network; it does not work within the "original" telephony network, which relies on standards such as SS7. More critically, although STIR/SHAKEN is frequently described as a "caller ID authentication technology," it does not authenticate any caller ID per se—instead, it authenticates the originating carrier, which solves a different problem and can cause security failures where the receiver accepts a digitally signed call as genuine but the caller ID is illegitimately spoofed. The reliance on trusted third parties also severely limits STIR/SHAKEN's ability to work across borders.For ChatCatHat™, compliance with the STIR/SHAKEN framework is a critical engineering constraint.This requires that the virtual numbers used by the system be legitimately provisioned and that the originating VoIP carrier can provide at minimum a B-level (Partial) attestation, confirming that the call originates from an authenticated source even though the calling number belongs to the middleware's virtual identity pool rather than the end-user directly.

2.3.3 PRC: CSL,DSL and PIPL

The Cybersecurity Law (CSL), the Data Security Law (DSL), and the Personal Information Protection Law (PIPL) represent three pillars of the Chinese data protection legislation system and together form an overarching framework governing data processing and cybersecurity issues.The CSL, effective since June 2017, is the first comprehensive legislation forming the backbone of data protection from a cyber security perspective, stipulating obligations for "network operators" and "critical information infrastructure operators" (CIIOs) in China. The DSL, adopted in 2021, further enhances data security by establishing a fundamental and categorized data security system applying to potentially all data processing activities, regardless of

Pic/2.11.png

Figure 2.11: The U.S. STIR/SHAKEN Standard Architecture

whether they are online or offline.

The PIPL was enacted on 20 August 2021 and came into effect on 1 November 2021; as China's first national-level legislation dedicated to personal information protection, it complements the CSL and the DSL. The PIPL applies mandatorily to the processing of personal information of natural persons within the territory of China, and its extraterritorial reach extends to activities carried out outside China that process personal information of individuals located in China for the purpose of providing products or services, or for analyzing or evaluating their behaviors. The law establishes robust individual rights over personal data and imposes stringent consent requirements: if there is no other legal basis for processing, handlers must obtain consent for data collection and processing, and this consent can be revoked by any individual at any time. PIPL establishes a spectrum of materially significant penalties for violations: confiscation of illegal gains, a fine of up to RMB 50 million (approximately USD 7.8 million) or 5 percent of the past year's turnover, and even the possibility of business suspension.Violations could also affect a company's social credit scores, making it difficult to access credit, purchase property, and conduct everyday business operations.

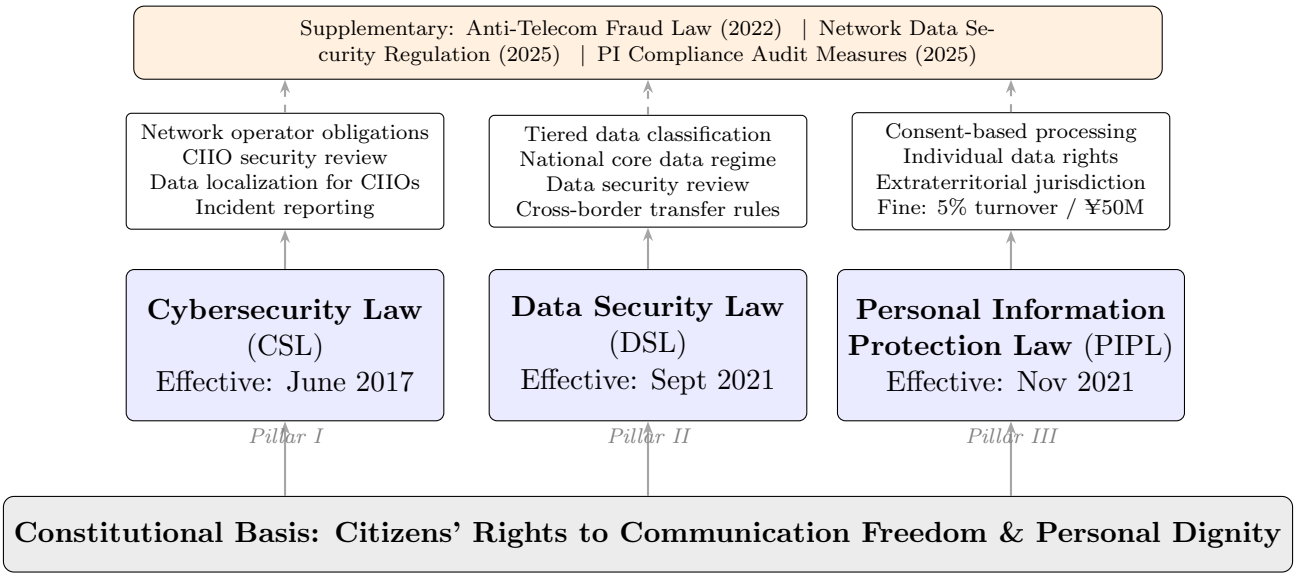

Figure 2.12: China's "Three Pillars" Data Protection Legislative Framework

2.3.4 ChatCatHat™: Far More Than a Simple Tool

The global regulatory landscape reveals a clear, converging trajectory: the right to communication privacy is being progressively elevated from a best-practice recommendation to a legally enforceable obligation, while simultaneously, regulations against caller ID spoofing and telecom fraud are being tightened. This apparent tension—between the right to privacy and the demand for caller ID authentication—defines the precise design space within which ChatCatHat must operate.

By ensuring that the virtual numbers presented to callees are legitimately provisioned, properly attested under frameworks like STIR/SHAKEN, and managed through auditable number-pool allocation policies, ChatCatHat achieves caller identity protection without resorting to the kind of outright spoofing that anti-fraud regulations target. The caller's real identity is concealed from the callee and from passive metadata surveillance, but remains traceable within the system's internal audit logs—satisfying both the GDPR's data minimization and purpose limitation principles and the lawful interception obligations that most jurisdictions impose on communication service providers.

ChatCatHat is not merely a privacy tool operating in a regulatory vacuum; rather, it is an engineering response to a regulatory environment that simultaneously demands stronger encryption, better identity protection, and continued lawful

accountability. The system's compliance-by-design approach—encrypting the caller-side VoIP link, anonymizing metadata at the middleware layer, and delivering calls via legitimately attested PSTN channels—positions it at the intersection of these converging regulatory mandates, offering a technically viable path toward reconciling privacy rights with public safety obligations.

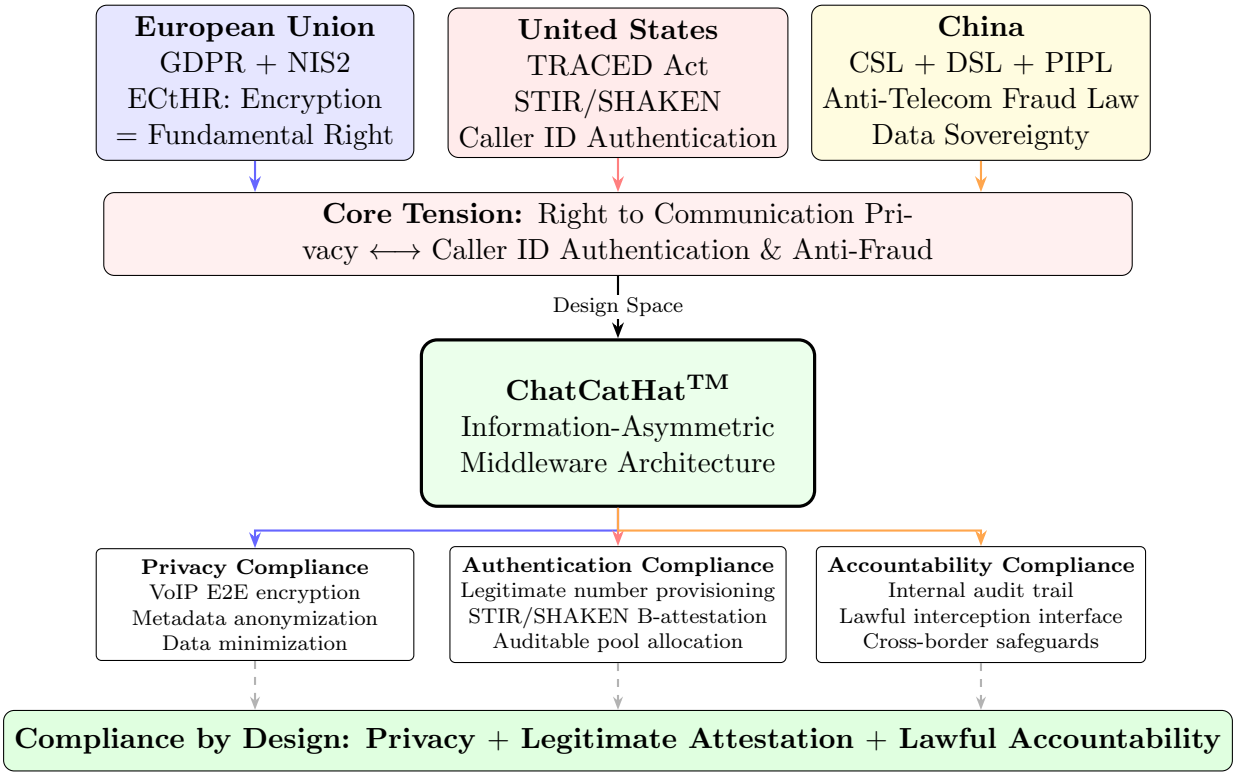

Figure 2.13: Regulatory Convergence and ChatCatHat™ Compliance-by-Design Positioning

Chapter 3

Competitive Analysis and Requirement Analysis

3.1 Competitive Analysis

The communication privacy landscape encompasses a diverse ecosystem of solutions,each addressing different facets of the caller-identity-versus-privacy tension-identified in Chapter 2. To rigorously position ChatCatHat™within this landscape, we categorize existing solutions into three architecturalfamilies: *pure VoIP* platforms, *pure PSTN virtual-number* services,and *AXB relay* (O2O intermediary) platforms. For each category, we select representative products, analyze their strengths and limitations, and contrast them with the ChatCatHat information-asymmetric middleware architecture.

3.1.1 Category I: Pure VoIP Platforms

Pure VoIP platforms provide end-to-end encrypted voice and messaging over IP networks, bypassing the PSTN entirely. Representative products include Signal, WhatsApp, and Telegram.

Signal

Developed by the non-profit Signal Technology Foundation, Signal is widely regarded as the "gold standard" in the realm of privacy-preserving communication. It employs the Signal Protocol to implement end-to-end encryption by default for all message types—including text, voice, and video. Beyond content encryption, Signal also implements the "Sealed Sender" mechanism—a metadata protection feature that conceals sender information even from Signal's own servers. The application collects minimal data: registration requires only a user's phone number, and no additional device information, usage data, or behavioral data is retained. Signal's entire codebase is open source and undergoes regular independent security audits—a level of verifiable transparency that proprietary software cannot match.

However, Signal's privacy model is architecturally constrained by its VoIP-only communication paradigm. Both parties to a conversation must have the Signal appli-

cation installed, thereby creating a "closed network" limitation. Signal calls cannot interface with the Public Switched Telephone Network (PSTN); consequently, Signal users cannot use the Signal network to call landlines, mobile numbers that do not have the app installed, or emergency services. This fundamentally limits its utility in scenarios requiring interaction with the broader telephone ecosystem—precisely the use case that ChatCatHat is designed to address.

Pic/3.1.png

Figure 3.1: Signal Messaging Architecture

WhatsApp

WhatsApp, owned by Meta (formerly Facebook), commands approximately 3 billion monthly active users globally, making it the most widely adopted encrypted messaging platform. It uses the same Signal Protocol for end-to-end encryption of message content. However, WhatsApp diverges significantly from Signal in its data practices: it collects extensive metadata including device details, IP addresses, usage patterns, and contact frequency. This metadata is shared with Meta's broader advertising ecosystem and can be disclosed to law enforcement upon request. In 2024, Meta reportedly complied with over 78% of U.S. law enforcement data requests involving WhatsApp metadata.

Like Signal, WhatsApp operates exclusively within its own VoIP network. Cross-network calling to PSTN endpoints is not supported, and the platform's proprietary (non-open-source) client prevents independent verification of its security claims beyond the encryption protocol layer.

Telegram

Telegram occupies an intermediate position: its standard Cloud Chats use client-server encryption (not E2E), while the optional Secret Chats feature provides E2E encryption using the MTProto 2.0 protocol.

Pic/3.4.png

Figure 3.2: WhatsApp

Voice calls in Telegram support E2E encryption, but only for one-to-one calls. Telegram's cloud-based architecture stores message history on its servers, enabling multi-device synchronization but creating a centralized data repository that represents a single point of compromise. Like Signal and WhatsApp, Telegram cannot originate or terminate calls on the PSTN.

Pic/3.3.png

Figure 3.3: Telegram

The fundamental limitation shared by all pure VoIP platforms is their inability to bridge into the PSTN. While they excel at protecting communication *within* their

Table 3.1: Pure VoIP Platforms: Comparative Summary

Criterion	**Signal**	**WhatsApp**	**Telegram**
E2E Encryption (default)	✓	✓	✗[a]
Metadata Protection	Sealed Sender	✗(Meta ecosystem)	✗
Open Source	Full client + protocol	Protocol only	Client only
PSTN Interoperability	✗	✗	✗
Caller ID on PSTN	N/A	N/A	N/A
STIR/SHAKEN Attestation	N/A	N/A	N/A
User Base (MAU)	∼40–70 M	∼3 B	∼900 M
Data Collection	Phone number only	Extensive	Moderate
Regulatory Compliance	Minimal (non-profit)	GDPR fines history	Mixed

[a] E2E available only in "Secret Chats"; not default for groups or voice.

walled gardens, they cannot solve the problem ChatCatHat addresses: enabling a privacy-protected caller to reach *any* telephone number worldwide while presenting a legitimate,STIR/SHAKEN-attested caller ID.

3.1.2 Category II: Pure PSTN Virtual-Number Services

This category encompasses services that provide users with secondary telephone numbers operating on the PSTN, typically via VoIP backends but delivering calls through traditional telephone infrastructure.

Google Voice

Google Voice provides users with a free, permanent U.S. virtual telephone number integrated into the Google ecosystem. It supports call forwarding, voicemail transcription, and SMS, all accessible through web and mobile interfaces. However, Google Voice is tightly bound to the user's Google account, requiring U.S. phone number verification at registration. It is not designed for temporary anonymity; rather, it functions as a permanent second line. Furthermore, as a Google product, all communication metadata is subject to Google's data practices and may be disclosed to law enforcement. Voice calls through Google Voice lack end-to-end encryption, and the service is geographically restricted to the United States.

Pic/3.2.png

Figure 3.4: Google Voice

Hushed

Hushed is a VoIP-based virtual number application offering temporary and long-term phone numbers across the U.S., Canada, and the U.K. It differentiates itself from competitors by not requiring a pre-existing cellular number at registration, providing a stronger anonymity baseline. Hushed supports call forwarding, voicemail, and auto-reply features, with plans ranging from pay-as-you-go ($1.99 for 7 days) to unlimited monthly subscriptions ($4.99/month). The service accepts cryptocurrency payments (Bitcoin, Ethereum), further supporting anonymous acquisition.

However, Hushed numbers are VoIP-originated and frequently identified as such by receiving carriers. This means that Hushed calls often receive STIR/SHAKEN B- or C-attestation at best, leading to potential spam labeling or blocking. The service provides no end-to-end encryption for voice calls—audio traverses Hushed's servers in cleartext or with transport-layer encryption only. Additionally, Hushed has documented reliability issues, with users reporting missed notifications and inconsistent call quality.

Burner

Burner provides disposable phone numbers with a focus on temporary use cases such as online dating, classified advertisements, and short-term business interactions. Unlike Hushed, Burner requires a pre-existing personal phone number for account verification, reducing its anonymity profile. Burner offers richer third-party

integrations (Slack, Dropbox, Google Drive) and advanced features including robo-call blocking. However, its geographic coverage is limited to the U.S. and Canada, and like Hushed, it provides no end-to-end encryption for the voice path.

Category Synthesis

Table 3.2: PSTN Virtual-Number Services: Comparative Summary

Criterion	**Google Voice**	**Hushed**	**Burner**
E2E Encryption	✗	✗	✗
Metadata Protection	✗(Google ecosystem)	Partial	Partial
Anonymous Registration	✗(Google account)	✓(no SIM)	✗(requires cell #)
PSTN Interoperability	✓	✓	✓
STIR/SHAKEN Attestation	A (Google carrier)	B/C (VoIP flag)	B/C (VoIP flag)
Geographic Coverage	U.S. only	US/CA/UK + 40 countries	U.S. & Canada
Disposable Numbers	✗($10 change fee)	✓	✓
Crypto Payment	✗	✓	✗
Call Quality	Good	Variable	Good

The critical weakness of PSTN virtual-number services is the absence of end-to-end encryption. The service provider has full access to call audio and metadata, creating a centralized trust dependency. Furthermore, VoIP-originated numbers increasingly suffer from degraded caller-ID trust under the STIR/SHAKEN framework, as most providers can only achieve B-attestation ("identity known, authorization to use number: unknown") or C-attestation ("gateway only") for their numbers.

3.1.3 Category III: AXB Relay Platforms (O2O Virtual Number Intermediaries)

AXB (A-calls-X, X-calls-B) relay platforms assign a temporary virtual number (X) to mediate communication between two parties (A and B), typically in marketplace or on-demand service contexts. The virtual number masks both parties' real numbers from each other.

Twilio Proxy

Twilio, the dominant Communications Platform as a Service (CPaaS) provider, offers a dedicated Proxy product for number masking. Twilio Proxy intelligently manages phone number selection from a configurable pool, supporting geo-matching, sticky and non-sticky allocation policies, and session-based lifecycle management. The platform provides virtual numbers in 100+ countries with programmatic APIs for session creation, participant management, and call routing. Twilio numbers provisioned directly through the platform can achieve STIR/SHAKEN A-attestation when used by verified enterprise customers.

However, Twilio Proxy is designed as a *business infrastructure* tool, not a *privacy* tool. The platform operator (the business using Twilio's API) has complete visibility into both parties' real numbers, all call metadata, and optionally call recordings. There is no encryption between participants beyond standard TLS transport security. Furthermore, Twilio's regulatory compliance model requires end-user identity verification for number provisioning in most jurisdictions, making anonymous individual use impractical.

Exotel ExoBridge (India)

Exotel, a prominent Indian cloud telephony provider, offers ExoBridge for virtual number management in marketplace contexts. It enables platforms to bridge buyers and sellers, agents and customers using temporary virtual numbers with configurable session durations. ExoBridge supports click-to-call integration, CRM bridging, and automated number pool management. The platform is widely deployed in Indian O2O ecosystems (food delivery, ride-hailing, e-commerce).

ExoBridge shares the same architectural limitation as Twilio Proxy: the platform operator retains full visibility into all communication metadata. Privacy is provided only *between* the two communicating parties, not *from* the platform itself. End-to-end encryption is not supported on the voice path.

Chinese Telecom AXB Services (China Mobile / China Unicom / China Telecom)

In China, the three major carriers (China Mobile, China Unicom, China Telecom) offer standardized AXB virtual-number relay services primarily consumed by O2O platforms such as Didi (ride-hailing), Meituan (food delivery), and various e-commerce marketplaces. These carrier-grade services provide high reliability, legitimate number attestation, and compliance with China's Anti-Telecom Fraud Law (2022). The virtual numbers are provisioned from carrier-owned pools, ensuring full regulatory traceability.

However, carrier AXB services are exclusively B2B products requiring enterprise contracts and real-name registration. Individual privacy users cannot directly access these services. Moreover, the carrier retains complete CDR (Call Detail Record) visibility, and under Chinese law, this data must be retained and made available to authorities upon request, with no end-to-end encryption on the voice path.

Category Synthesis

Table 3.3: AXB Relay Platforms: Comparative Summary

Criterion	**Twilio Proxy**	**Exotel ExoBridge**	**CN Carrier AXB**
E2E Encryption	✗	✗	✗
Metadata Protection	✗(operator visible)	✗	✗
Individual Access	✗(B2B API)	✗(B2B)	✗(B2B)
PSTN Interoperability	✓	✓	✓
STIR/SHAKEN Attestation	A (direct) / B (reseller)	N/A (India)	N/A (China)
Number Pool Management	Programmatic API	Automated	Carrier-managed
Session Lifecycle	Configurable	Configurable	Platform-defined
Regulatory Compliance	Strong (multi-jurisdiction)	India-focused	China-specific

3.1.4 Cross-Category Competitive Positioning of ChatCatHat

Figure 3.5 illustrates ChatCatHat's unique positioning at the intersection of the three categories analyzed above.

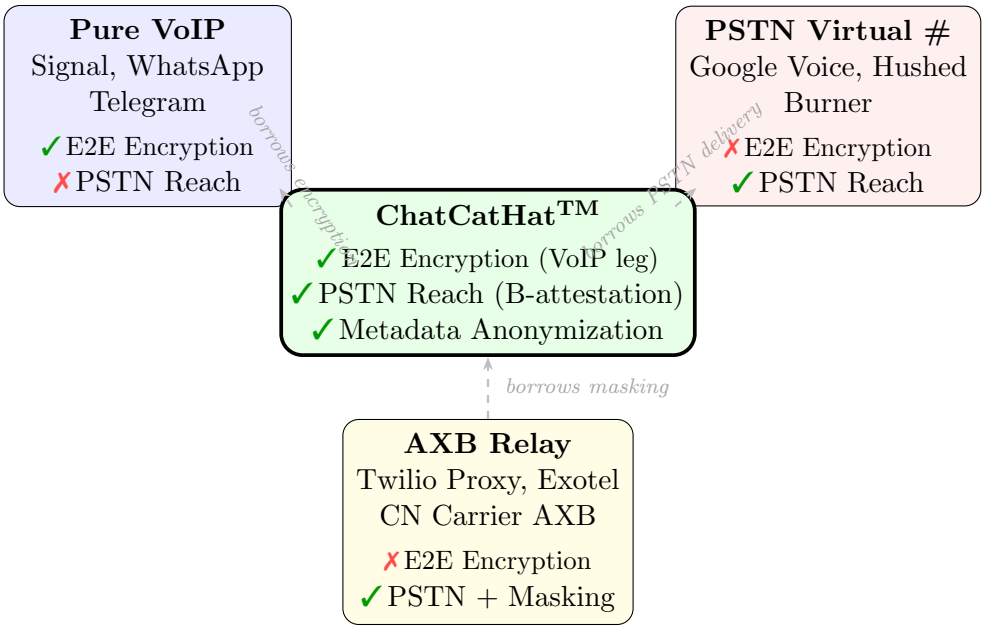

Figure 3.5: Cross-Category Competitive Positioning of ChatCatHat™

The key insight from this competitive analysis is that no existing product category simultaneously provides all three properties that ChatCatHat targets:

1. **End-to-end encryption** on the caller-side VoIP link (borrowed from the pure VoIP paradigm);

2. **Legitimate PSTN termination** with STIR/SHAKEN B-attestation (borrowed from the virtual-number paradigm);

3. **Information-asymmetric metadata anonymization** where the middleware itself cannot correlate caller identity with callee identity (a novel contribution beyond the AXB relay paradigm).

Table 3.4 provides the consolidated cross-category comparison.

The competitive landscape reveals a structural trilemma that has remained unresolved across all three existing product categories: a platform may offer *strong encryption* (pure VoIP), *universal PSTN reachability* (virtual-number services), or *identity masking with carrier-grade trust* (AXB relays)—but no incumbent solution delivers all three simultaneously, and none attempts to eliminate platform-side omniscience over session metadata. ChatCatHat™ is architecturally designed to break this trilemma. By fusing an SRTP-encrypted VoIP ingress leg with a

Table 3.4: Grand Competitive Comparison: ChatCatHat vs. All Categories

Property	**Pure VoIP**	**PSTN V-Num**	**AXB Relay**	**ChatCatHat**
VoIP E2E Encryption	✓	✗	✗	✓
PSTN Call Termination	✗	✓	✓	✓
Number Masking	N/A	Partial	✓	✓
Metadata Anonymization	✓[a]	✗	✗	✓
STIR/SHAKEN Attestation	N/A	B/C	A/B	B
Individual User Access	✓	✓	✗	✓
No Platform Omniscience	✓	✗	✗	✓[b]
Lawful Interception Path	Contested	✓	✓	✓
Cross-Jurisdiction Design	Partial	Partial	Partial	✓

[a] Signal only (Sealed Sender). [b] Via information-asymmetric architecture; see §3.2.

STIR/SHAKEN-attested PSTN egress leg, mediated through an information-asymmetric middleware whose subsystems are cryptographically isolated from one another, the system occupies a genuinely novel point in the design space. It inherits the content-protection strengths of Signal, the telephone-network universality of Hushed and Google Voice, and the caller-ID-masking utility of Twilio Proxy—while introducing the additional, and critically differentiating, guarantee that even the platform operator itself cannot reconstruct the full $\langle \text{caller}, \text{callee} \rangle$ mapping for any given session without a cryptographically authorized, legally mandated re-assembly process. This positioning makes ChatCatHat not merely an incremental improvement over any single competitor, but a *category-bridging* architecture that redefines the attainable privacy–reachability–trust frontier for telephony middleware.

3.2 Requirements Analysis

Building upon the problem statement established in Chapter 1 and the competitive gaps identified in §3, this section develops a comprehensive requirements specification for ChatCatHat™. The specification follows a structured methodology inspired by the IEEE 830 standard for software requirements documents, adapted to accommodate the dual nature of the system as both a real-time media platform and a privacy-preserving middleware.

Requirements are organized into five categories: functional requirements (§3.2.1),

non-functional requirements (§3.2.2), security requirements (§3.2.3), privacy requirements (§3.2.4), and regulatory compliance requirements (§**??**). Each requirement is assigned a priority level using the MoSCoW convention—*Must*, *Should*, or *May*—to distinguish between non-negotiable constraints, strongly recommended features, and optional enhancements that may be deferred to later development phases. The traceability of these requirements to the competitive deficiencies catalogued in §3 is maintained throughout: where a requirement directly addresses a gap left by an incumbent product category, the relevant competitive context is noted in the accompanying discussion.

3.2.1 Functional Requirements

Functional requirements define the observable behaviors the system must exhibit. They are organized by subsystem boundary, reflecting the information-asymmetric architecture that is central to ChatCatHat's design philosophy. This decomposition is not merely an organizational convenience; it is itself a requirement-level decision, since the isolation between subsystems is what enables the privacy guarantees formalized later in §3.2.4. Each subsystem is specified to be independently deployable and independently auditable, so that a compromise or subpoena directed at one subsystem does not automatically grant access to the data held by another.

FR-1: VoIP Client Subsystem

The VoIP client is the sole user-facing component of the ChatCatHat architecture. It serves a dual purpose: first, it provides a conventional telephony user experience—dialing, ringing, in-call audio, DTMF interaction—so that the privacy-enhancing nature of the system remains transparent to non-technical users; second, it acts as the cryptographic anchor for the caller-side encryption chain, ensuring that the SRTP session key is negotiated directly between the client and the middleware's media ingress point without traversal through any intermediate proxy that could perform key escrow.

Table 3.5: Functional Requirements: VoIP Client

ID	Priority	Description
FR-1.1	Must	The client SHALL establish an encrypted VoIP session (SRTP with DTLS-SRTP or ZRTP key exchange) to the ChatCatHat middleware upon call initiation.
FR-1.2	Must	The client SHALL support DTMF relay (RFC 4733) for IVR interaction on the PSTN-side callee's system.
FR-1.3	Must	The client SHALL allow the user to input any valid E.164 telephone number as the callee destination.
FR-1.4	Should	The client SHOULD support codec negotiation including Opus, G.711 μ-law, and G.722 for quality adaptation.
FR-1.5	Should	The client SHOULD display the assigned outbound caller ID (virtual number) to the user before call placement.
FR-1.6	May	The client MAY support in-app contact management with locally-encrypted storage (never synced to server).

FR-2: Middleware (Information-Asymmetric Core)

The middleware is the architectural heart of ChatCatHat and the component most directly responsible for the system's differentiation from existing products. Functionally, it must perform the role of a Back-to-Back User Agent (B2BUA) in SIP terminology: it terminates the inbound VoIP session from the client, applies media processing (transcoding, gain normalization, comfort noise generation), and originates an entirely new outbound SIP session toward the carrier partner's trunk. Crucially, the two legs are *logically independent* at the signaling level—the SIP headers, SDP bodies, and RTP streams on the PSTN-facing leg bear no residual trace of the caller-facing leg's identifiers.

Metadata stripping (FR-2.4) deserves particular emphasis, because it addresses the most common failure mode of privacy in existing VoIP systems. Studies of SIP-based VoIP deployments have shown that even when media is encrypted, the SIP signaling layer routinely leaks the caller's real IP address (via the Via and Contact headers), SIP URI (which often encodes a username or phone number), User-Agent string (which fingerprints the client software and device), and timing information (which enables traffic-analysis attacks). ChatCatHat's middleware is required to sanitize all such fields before the outbound SIP INVITE is constructed. The outbound leg's SIP headers are generated *de novo* by the middleware, populated exclusively with the virtual number's identity, the middleware's own IP address, and a generic User-Agent string.

Call-back routing (FR-2.5) addresses a practical usability concern that has historically limited the adoption of number-masking services: if the callee sees an unfamiliar virtual number and calls it back, the return call must be routed to the original caller within a reasonable time window, or the system breaks the social contract of telephony reciprocity. Implementing this requires the middleware to maintain a short-lived session mapping—associating the virtual number with the original caller's VoIP endpoint—which creates a tension with the metadata minimization requirement (PRV-3). The resolution, elaborated in the design chapter, involves storing this mapping in a volatile, TTL-bounded cache that is automatically purged and is accessible only to the routing subsystem, never to the authentication or number-management subsystems.

Table 3.6: Functional Requirements: Middleware Core

ID	Priority	Description
FR-2.1	Must	The middleware SHALL terminate the inbound VoIP/SRTP session and originate a new PSTN call leg via SIP trunk to the carrier partner, performing media transcoding as needed.
FR-2.2	Must	The middleware SHALL select an outbound virtual number from the allocated pool and insert it as the caller ID on the PSTN leg.
FR-2.3	Must	The middleware SHALL implement session state management including call setup, in-call DTMF forwarding, and graceful teardown.
FR-2.4	Must	The middleware SHALL perform metadata stripping: the caller's real IP address, SIP URI, and device fingerprint SHALL NOT be forwarded to the PSTN leg or stored in long-term logs.
FR-2.5	Should	The middleware SHOULD support call-back routing: if the callee calls back the virtual number within a configurable TTL, the call SHALL be routed to the original caller.
FR-2.6	Should	The middleware SHOULD implement virtual number rotation policies (per-call, per-session, or sticky allocation) configurable by the user.
FR-2.7	May	The middleware MAY support SMS relay through the assigned virtual number, subject to the same privacy guarantees.

FR-3: Number Pool Management

The number pool is the finite, regulated resource upon which the entire masking capability depends. Unlike purely software-defined resources (compute, memory, bandwidth) that can be provisioned elastically, telephone numbers are subject to regulatory allocation frameworks, carrier contracts, and STIR/SHAKEN attestation requirements that introduce hard constraints on availability, cost, and geographic coverage.

Collision avoidance (FR-3.2) is a correctness requirement with privacy implications. If two concurrent sessions directed at the same callee (or at two callees who share a household) are assigned the same virtual number, the callee could infer that the two calls originate from the same service, or even correlate the calls if additional context is available. The allocation engine must therefore maintain a real-time occupancy map of the number pool, indexed by both virtual number and callee number, and enforce mutual exclusion at the assignment level.

Table 3.7: Functional Requirements: Number Pool Management

ID	Priority	Description
FR-3.1	Must	The system SHALL maintain a pool of legitimately provisioned telephone numbers with auditable provenance (carrier LOA on file).
FR-3.2	Must	Number allocation SHALL ensure no collision: two concurrent sessions with overlapping callee numbers SHALL NOT be assigned the same virtual number.
FR-3.3	Should	The system SHOULD support geo-matching: the assigned virtual number SHOULD share the same country code (and preferably area code) as the callee.
FR-3.4	Should	The system SHOULD support automatic pool scaling based on concurrent session demand, with carrier API integration for dynamic provisioning.
FR-3.5	May	The system MAY implement number health monitoring (STIR/SHAKEN attestation verification, spam-score checking) and automatically retire flagged numbers.

FR-4: User Account and Authentication

The account subsystem embodies one of ChatCatHat's most philosophically distinctive positions: that a privacy-preserving telephony service should not require the user to surrender the very identity they are trying to protect as a precondition of enrollment. Virtually every competing product—Google Voice, Hushed, Burner, and all carrier-grade AXB services—requires a real telephone number or government-issued identity document at registration, thereby creating a single point of de-anonymization that can be compelled by court order, compromised by data breach, or exploited by an insider threat. ChatCatHat's FR-4.1 explicitly breaks this dependency by supporting anonymous registration via email alias or cryptographic identity (e.g., a public key).

Token-based authentication (FR-4.2) is specified rather than session-cookie or basic-auth alternatives for two reasons. First, JWT tokens are stateless and can be

verified by any middleware node without a centralized session store, which aligns with the horizontal scaling requirement (NFR-4). Second, the token's claims structure allows fine-grained authorization decisions (e.g., call quota remaining, number rotation policy preference, tier level) to be encoded directly in the token, reducing the need for synchronous database lookups during the latency-sensitive call setup path.

Table 3.8: Functional Requirements: User Account

ID	Priority	Description
FR-4.1	Must	User registration SHALL NOT require a personal telephone number or government ID; anonymous registration via email alias or cryptographic identity is supported.
FR-4.2	Must	The system SHALL implement token-based authentication (e.g., JWT with short-lived access tokens and rotating refresh tokens).
FR-4.3	Should	The system SHOULD support optional two-factor authentication (TOTP-based, not SMS-based).
FR-4.4	May	The system MAY support cryptocurrency-based payment for subscription and usage fees to minimize financial identity linkage.

Figure 3.6 provides a high-level functional decomposition summarizing the four subsystems and their external interfaces. The dashed lines connecting each subsystem to its primary external actor underscore a key architectural invariant: each subsystem communicates with exactly one class of external entity, and the data exchanged across that boundary is scoped to the subsystem's responsibility alone. The VoIP client subsystem interfaces only with the caller's device; the middleware interfaces only with the PSTN network; the number pool subsystem interfaces only with carrier provisioning APIs; and the account subsystem interfaces only with the payment gateway. No subsystem holds a cross-cutting view of the end-to-end call path, which is the structural precondition for the information-asymmetric privacy guarantees formalized in §3.2.4.

3.2.2 Non-Functional Requirements

Non-functional requirements constrain *how* the system delivers its functionality, rather than *what* functionality it delivers. For a real-time media system like ChatCatHat, non-functional requirements are not secondary concerns—they are existential. A privacy-preserving telephony middleware that introduces perceptible audio

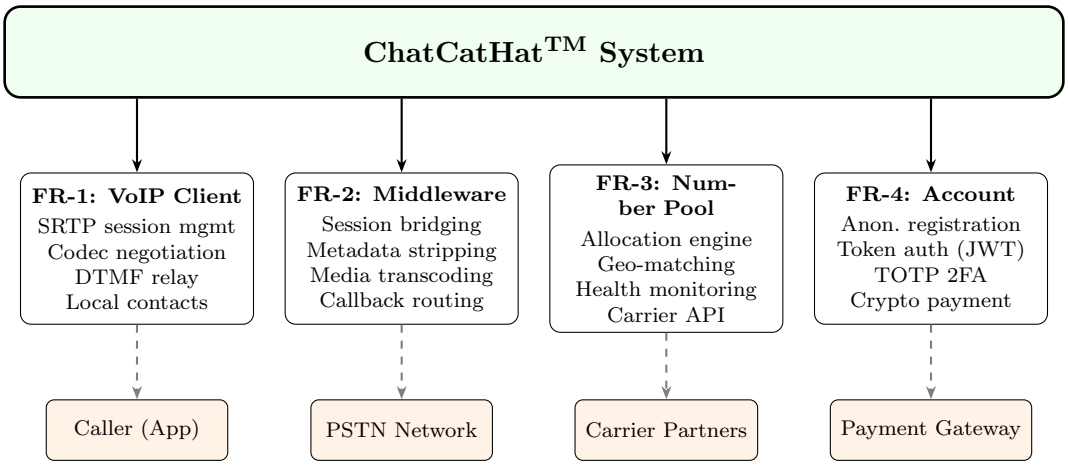

Figure 3.6: Functional Decomposition of ChatCatHat™

delay, drops calls under moderate load, or fails to interoperate with standard PSTN signaling protocols will be abandoned by users regardless of the sophistication of its privacy architecture. The requirements in this subsection are therefore specified with quantitative thresholds wherever possible, to enable objective verification during acceptance testing.

Latency (NFR-1) is the single most user-perceptible quality metric in telephony. The ITU-T G.114 recommendation establishes 150 ms one-way mouth-to-ear delay as the threshold for "excellent" quality and 400 ms as the upper bound of acceptability. ChatCatHat's architecture inherently introduces additional latency sources that do not exist in direct PSTN-to-PSTN calls: the VoIP-to-middleware network hop, SRTP decryption, potential codec transcoding (e.g., Opus to G.711), re-encryption or re-packetization for the PSTN leg, and the SIP trunk hop to the carrier partner. The 250 ms hard ceiling and 150 ms target specified in NFR-1 are therefore aggressive constraints that will require careful optimization of the media pipeline, including pre-allocated transcoding buffers, jitter buffer tuning, and co-location of middleware nodes with carrier SIP trunk ingress points.

Throughput and scalability (NFR-2, NFR-4) are specified together because they are architecturally coupled. The minimum of 1,000 concurrent sessions per node establishes the per-unit capacity requirement, while the horizontal scaling requirement establishes the growth model. The stateless design mandated by NFR-4—with session state externalized to a distributed store such as Redis Cluster—ensures that new media processing nodes can be added to the cluster without re-partitioning

existing sessions. A SIP-aware load balancer (e.g., Kamailio or OpenSIPS in dispatcher mode) is required rather than a generic HTTP load balancer, because SIP session affinity must be maintained across the INVITE/200 OK/ACK three-way handshake and subsequent in-dialog requests (re-INVITE, BYE) for the duration of the call.

Table 3.9: Non-Functional Requirements

ID	Category	Priority	Description
NFR-1	Latency	Must	End-to-end voice latency (mouth-to-ear) SHALL NOT exceed 250 ms under normal network conditions, with a target of ≤150 ms. The middleware processing overhead SHALL contribute ≤30 ms.
NFR-2	Throughput	Must	The middleware SHALL support a minimum of 1,000 concurrent call sessions per node, with horizontal scaling capability to 10,000+ sessions via load-balanced clustering.
NFR-3	Availability	Must	The system SHALL achieve 99.95% uptime (annualized), measured at the middleware API endpoint, excluding scheduled maintenance windows.
NFR-4	Scalability	Should	The architecture SHALL support stateless horizontal scaling of media processing nodes behind a SIP-aware load balancer, with session state externalized to a distributed store (e.g., Redis Cluster).
NFR-5	Audio Quality	Should	Mean Opinion Score (MOS) for transcoded calls SHALL be ≥3.8 (on a 1–5 scale) as measured by PESQ/POLQA objective metrics.
NFR-6	Portability	Should	The VoIP client SHALL support iOS 16+, Android 12+, and a WebRTC-based browser client (Chrome, Firefox, Safari).
NFR-7	Interoperability	Must	The PSTN egress path SHALL comply with SIP (RFC 3261), SRTP (RFC 3711), and STIR/SHAKEN (RFC 8224/8225/8226) standards.
NFR-8	Maintainability	Should	The middleware codebase SHALL achieve ≥80% unit test coverage, with automated CI/CD pipelines for deployment.

3.2.3 Security Requirements

Security requirements address threats to the system's integrity, confidentiality, and availability. They are derived from the threat model implicit in the problem statement: the adversary may include passive network observers (e.g., ISP-level traffic inspection or state-level surveillance), compromised intermediate nodes (e.g., a rogue SIP proxy in the signaling path), active attackers targeting the middleware infras-

tructure (e.g., SIP scanning, toll fraud, telephony denial of service), and insider threats (e.g., a compromised or coerced system administrator). The security architecture adopts a defense-in-depth strategy, layering complementary controls from the network transport level up through application logic and governance processes, as illustrated in Figure 3.7.

The VoIP connection between the client and middleware is the system's most critical security boundary, as it transmits sensitive voice data over the public internet. To prevent interception and man-in-the-middle attacks, the system employs high-standard encryption: SRTP (AES-256-GCM) for the media channel and SIP over TLS 1.3 for signaling. Crucially, it uses mutual TLS (mTLS) with client certificates for authentication, completely eliminating the need for plaintext password transmission during call setup.

SEC-3 addresses the residual risk that arises from the middleware's role as a media transcoding bridge. Because the middleware must decode the incoming SRTP stream (to transcode from the VoIP codec to the PSTN codec) and re-encode it for the outbound leg, decrypted audio momentarily exists in the middleware's memory. SEC-3 constrains this exposure by mandating that the transcoding pipeline operate entirely in volatile memory, with explicit zeroing of all audio buffers upon session teardown. No decrypted media may be written to persistent storage (disk, database, or object store) under any circumstances. This requirement is auditable through memory-safe coding practices, runtime instrumentation, and periodic forensic analysis of the middleware's storage subsystem.

At the application layer, authentication relies on short-lived JWTs (15 mins) to minimize exposure windows, while RS256/EdDSA signatures allow scalable verification without distributing shared secrets. Rate limiting provides essential brute-force protection for anonymous accounts. At the SIP layer, a dedicated Intrusion Detection System (IDS) is required. Generic firewalls cannot parse SIP semantics, making a SIP-specific IDS necessary to block VoIP-centric threats like endpoint enumeration, toll fraud, and TDoS attacks.

Table 3.10: Security Requirements

ID	Priority	Description
SEC-1	Must	All VoIP signaling (SIP) between client and middleware SHALL be transported over TLS 1.3 with mutual certificate authentication.
SEC-2	Must	All VoIP media (RTP) between client and middleware SHALL be encrypted using SRTP with AES-256-GCM, with DTLS-SRTP or ZRTP key exchange to achieve end-to-end key agreement without middleware key escrow.
SEC-3	Must	The middleware SHALL NOT retain decrypted media content in any persistent storage. Media transcoding SHALL operate in a volatile memory-only pipeline with explicit zeroing upon session teardown.
SEC-4	Must	API authentication tokens SHALL use short-lived JWTs ($\leq$15 min expiry) with RS256 or EdDSA signatures. Refresh tokens SHALL be stored in secure, HttpOnly cookies with SameSite=Strict.
SEC-5	Must	The system SHALL implement rate limiting and progressive back-off on authentication endpoints to mitigate brute-force and credential-stuffing attacks.
SEC-6	Should	The middleware SHALL undergo annual penetration testing by an independent third party, with findings remediated within 30 days (critical) / 90 days (high).
SEC-7	Should	The system SHALL implement intrusion detection at the SIP layer, including detection of SIP scanning (SIPVicious-class), toll fraud attempts, and TDoS (Telephony Denial of Service) patterns.
SEC-8	Should	All inter-service communication within the middleware cluster SHALL use mTLS with short-lived certificates rotated via an internal PKI or service mesh (e.g., Istio/Linkerd).

3.2.4 Privacy Requirements

Privacy requirements are distinguished from security requirements in that they address the *information-theoretic* properties of the system's data flows, not merely the cryptographic protection of those flows. A system can be perfectly secure—all channels encrypted, all credentials strong, all patches current—and yet fundamentally privacy-violating if the operator itself aggregates a comprehensive record of who called whom, when, and for how long. This is precisely the situation with incumbent AXB platforms (Twilio Proxy, Exotel, carrier AXB), where the platform operator holds the complete session mapping in plaintext as an operational necessity. The core privacy innovation of ChatCatHat—the information-asymmetric architecture—is formalized in this subsection as a set of verifiable requirements.

ChatCatHat's core innovation lies in the recognition that "security does not equate to privacy" (as traditional encryption systems may still allow operators to log all call metadata). To address this, the system employs an asymmetric infor-

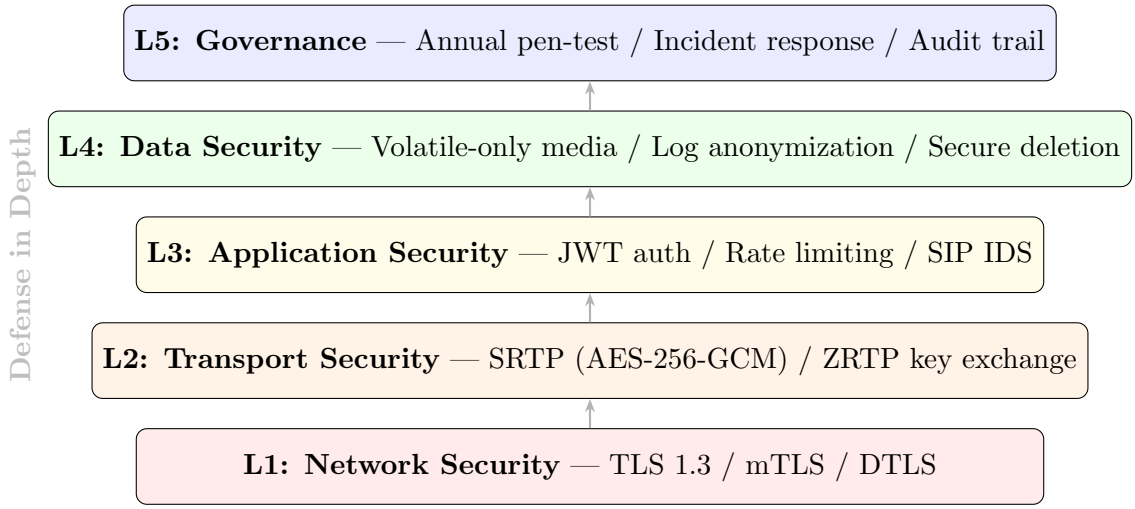

Figure 3.7: Security Requirements: Defense-in-Depth Layer Model

mation architecture (PRV-1) that strictly partitions data across three independent subsystems: the Authentication System knows only the "caller," the Routing System possesses only the "call parameters," and the Number Management System knows only the "callee." No single node can simultaneously piece together the true identities of both the caller and the callee. Even in the event of a judicial subpoena, reconstructing the records requires cross-system collaboration—much like a bank's "multi-signature authorization" protocol—thereby fundamentally preventing data misuse by the platform provider. At the level of individual calls, the system simultaneously guards against privacy threats originating from both the call recipient and the platform itself. PRV-2 ensures that the called party not only sees only a virtual number but is also unable to deduce the caller's true identity through network fingerprinting techniques (such as SIP headers or codec characteristics). PRV-3 mandates that the platform retain only a minimal set of anonymized billing data, strictly prohibiting the storage of both parties' true identifiers within the same record. In conjunction with PRV-4 (Data Retention Limits), the system defaults to retaining data for a period of only 90 days (or the minimum duration permitted by law), thereby significantly mitigating the analytical risks associated with long-term data accumulation.

To counter long-term behavioral profiling, the system implements unlinkability (PRV-5) and identity forward secrecy (PRV-6). Each call generates a completely uncorrelated random ID and utilizes a rotating virtual number, ensuring that even

if an attacker were to compromise a specific call, they would be unable to trace it back to reconstruct that user's historical call records. Furthermore, the specifications propose the highest level of plausible deniability (PRV-7)—rendering it cryptographically impossible for the system to conclusively prove that a specific individual initiated a particular call. However, given the legal risks this feature poses regarding the potential obstruction of judicial investigations, it is currently designated as an optional area for future exploration.

Table 3.11: Privacy Requirements

ID	Priority	Description
PRV-1	Must	**Information Asymmetry:** The middleware architecture SHALL be designed such that no single subsystem or database holds sufficient information to simultaneously link a caller's real identity (SIP credentials, IP address) with the callee's real telephone number for any given session.
PRV-2	Must	**Caller Anonymity:** The callee SHALL NOT be able to determine the caller's real telephone number, IP address, geographic location, or device identity from any information transmitted during or after the call.
PRV-3	Must	**Metadata Minimization:** Call Detail Records retained by the system SHALL contain only: (a) anonymized session ID, (b) call timestamp (rounded to nearest hour), (c) call duration (rounded to nearest minute), and (d) virtual number used. Caller and callee identifiers SHALL NOT appear in the same record.
PRV-4	Must	**Data Retention Limitation:** Anonymized CDRs SHALL be automatically purged after a configurable retention period (default: 90 days), aligned with the shortest applicable regulatory minimum.
PRV-5	Should	**Unlinkability:** Successive calls by the same user SHALL NOT be linkable by an external observer through traffic analysis of virtual number patterns, timing, or volume, subject to the virtual number rotation policy (FR-2.6).
PRV-6	Should	**Forward Secrecy of Identity:** Compromise of a single session's metadata SHALL NOT enable reconstruction of the user's historical call graph.
PRV-7	May	**Plausible Deniability:** The system MAY support a mode where session records are cryptographically structured such that even under compelled disclosure, the operator cannot prove a specific caller placed a specific call.

Figure 3.8 illustrates the structural realization of PRV-1. The three subsystems—authentication, routing, and number management—are depicted with their respective data stores, each containing a disjoint subset of the session metadata. The red dashed lines represent isolation boundaries enforced at the infrastructure level. The critical observation is that no single data store contains both a caller identifier and a callee identifier: the authentication store holds user credentials but no callee data; the routing store holds session parameters but no caller identity; and the number store holds virtual-to-callee mappings but no caller data. Reconstructing the full

end-to-end mapping requires a coordinated query across all three stores, gated by a cryptographic re-assembly protocol that is designed to be auditable and judicially supervised.

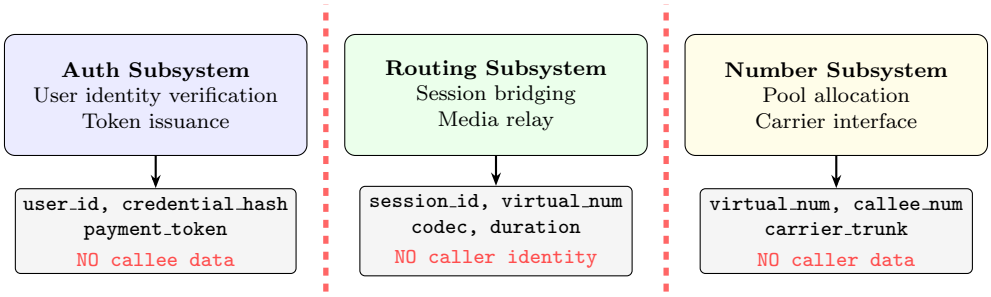

Figure 3.8: Information-Asymmetric Architecture: Subsystem Data Isolation

Chapter 4

System Design

To support the aforementioned theoretical model, the project adopts a thoroughly decoupled three-layer composite architecture consisting of the **Control Plane**, **Media Plane**, and **State Plane** in its engineering implementation.

Table 4.1: Three-Layer Decoupled Architecture of ChatCatHat™

Architecture Layer	**Core Tech Stack**	**Responsibilities & Engineering Value**
Control Plane	Node.js, Express 5	Handles API authentication, number-selection strategies (e.g. caller–callee consistency), business-flow orchestration, and Ops & Maintenance API exposure. The event-driven, non-blocking I/O nature of Node.js is inherently suited for signaling-control scenarios.
Media Plane	Asterisk[2], PJSIP[3], RTP	Serves as the underlying PBX core, dedicated to SIP signaling parsing, channel bridging (Bridge), ringing, and RTP audio media-stream relay. This decouples the media-computation overhead from the control plane.
State Plane	SQLite, better-sqlite3	Manages persistent data storage: user configurations, virtual-number pools, state-machine logs, and call-leg (A-leg / B-leg) granular flow-state transitions.

In the concrete call-flow process the system employs the **Back-to-Back User Agent (B2BUA)** pattern to achieve **Session Decoupling**:

- **Link Splitting:** The system decomposes the original point-to-point call into two independent logical chains. Flow A (S_C) is established between the caller

and the middleware; Flow B (S_T) is set up between the middleware and the callee.

- **Event-Driven Dialing:** The control-plane Node.js issues an `Originate` command via the AMI (Asterisk Manager Interface), driving media-plane Asterisk to first establish the A leg and then initiate the B leg call toward the public network with the virtual caller ID attached.

- **Seamless Stitching:** Upon the callee answering, the middleware immediately invokes the media-bridge function to join two independent media streams:

$$Connection = \mathcal{B}(S_C,\ S_T),$$

whereby both parties' voice data is securely relayed through the system.

Pic/3.5.png

Figure 4.1: Privacy Calling Server Architecture

4.1 Privacy Calling Architecture Design

4.1.1 Dual Callback Chain

When establishing a private call, the system employs the classic Double-Callback mechanism. This mechanism physically splits a single, complete end-to-end call into two independent call links (the A-leg and the B-leg), thereby achieving physical-level isolation of the actual numbers of both the calling and called parties at the

core switching nodes. The entire sequence of interactions can be divided into the following four key stages:

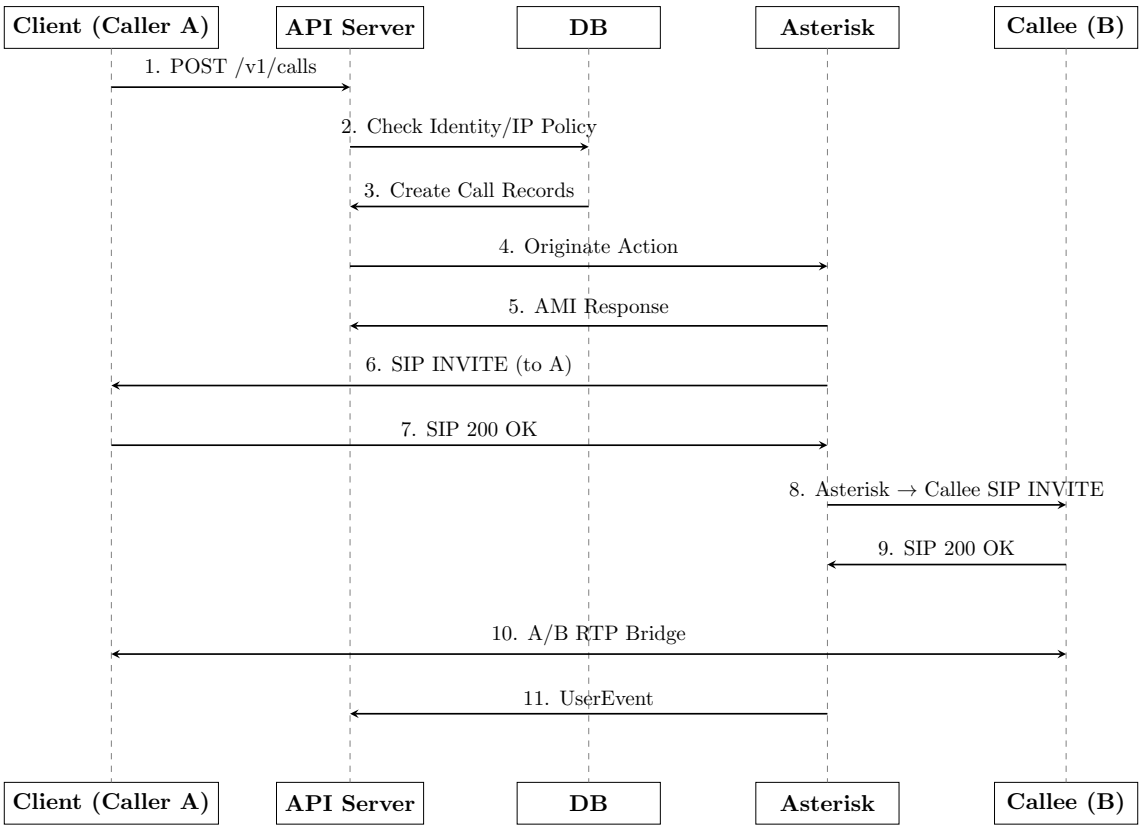

Figure 4.2: API-initiated call flow sequence diagram

- **Phase 1: Calling Initialization.** The calling client initiates a 'POST /v1/calls'request to the server API, with the request body containing the caller's identifier and the target callee's E.164 number. At the API layer, security authentication is first performed via a static API Key interceptor. Upon successful authentication, the system internally invokes the Policy Module (ϕ) to calculate and assign a suitable virtual number for the current session. Subsequently, the system inserts a primary record with a status of 'CREATED' into the 'CALLS' table within the SQLite database, while simultaneously initializing the pending states for both the A-leg and B-leg within the 'CALL_LEGS' table.

- **Phase 2: A_Leg Establishment.** The Control Plane utilizes an established, persistent TCP connection to issue an Originate action command to the Media Plane via the AMI (Asterisk Manager Interface) protocol. This command

contains the PJSIP address of the calling endpoint, as well as a preset timeout parameter. Upon receiving this action, the underlying SIP stack within Asterisk initiates the transmission of an INVITE request to the caller. At this juncture, the caller's mobile phone or softphone begins to ring. When the caller taps to answer, the endpoint returns a SIP 200 OK response, signaling that the A-leg media channel has been successfully established.

- **Phase 3: Privacy Context Bridging.** Once the A-leg is established, Asterisk's internal Dialplan automatically routes the call to a dedicated 'privacy_bridge' context. Within this context, the system's primary action is to perform Caller ID spoofing. Asterisk erases the caller's original number and replaces it with the virtual number assigned during Phase 1. Immediately thereafter, Asterisk sends a second SIP INVITE request to the callee, carrying this spoofed Caller ID. At this point, the Caller ID displayed on the callee's terminal is exclusively this virtual number, thereby completely masking the caller's true identity.

- **Phase 4: Media Session Connection.** Once the called party answers (responding with a SIP 200 OK), Asterisk's bridging engine activates, interconnecting the RTP media ports of the A-leg and B-leg to enable bidirectional voice flow. During this process, Asterisk encapsulates key points in the underlying signaling—such as Ringing, Answer, Bridge, and Hangup—as AMI UserEvent messages, which are then asynchronously pushed to the Node.js server. Upon receiving the Bridge event, the server updates the database status to BRIDGED, thereby completing a full closed loop spanning from the control plane down to the data plane callback.

4.1.2 Number Selection Strategies

In traditional random privacy number assignment schemes, called parties frequently receive incoming calls from varying numbers; this makes them highly susceptible to being flagged as spam calls, thereby leading to a decline in connection rates. To address this operational pain point, this system employs an Identity Selection Policy (ϕ) based on called-party consistency mapping. The core logic of this policy lies in

establishing and maintaining a long-term, weak binding relationship between the called party's number and the virtual number. The algorithm's execution flow is as follows:

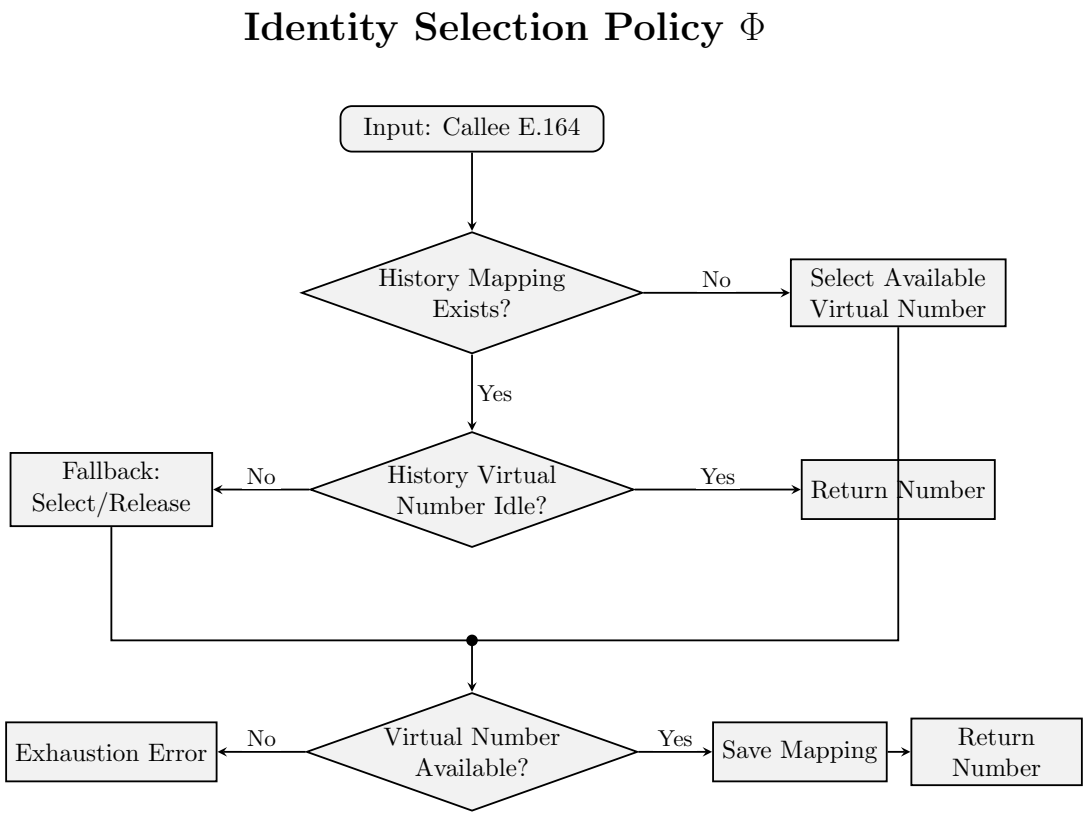

Figure 4.3: Identity Selection Policy Φ Flow Chart

Whenever a call is initiated, the system first extracts the callee's E.164 number and queries the ID_MAPPINGS table within the SQLite database to retrieve historical records. If a previously bound virtual number is found to exist and is currently in an Idle state, the system directly reuses that number to ensure answering consistency and foster a sense of trust for the callee. However, if that historical virtual number is currently occupied by another concurrent call—to guarantee high availability and prevent call blocking—the system triggers a conflict fallback mechanism, actively discarding the old mapping and proceeding to the new number allocation workflow. For calls lacking historical records or those for which the fallback mechanism has been triggered, the system immediately scans the global VIRTUAL_NUMBERS pool to allocate an unoccupied, idle number, and subsequently writes the new mapping relationship to the database. Finally, in the event of extreme concurrency leading to the complete depletion of the global number pool, the system directly propagates

a Virtual Number Exhaustion Error to the upper layer; this action halts the process and prevents the underlying Asterisk system from generating invalid signaling.

Table 4.2: Identity Selection Policy Φ Scheduling Scenarios

Policy Trigger Scenario	History Mapping Status	Target Virtual Number Concurrency Status	System Scheduling Behavior
First Outbound Call	No Record	N/A	Allocate a new number from the number pool and create a new mapping in the `ID_MAPPINGS` table.
Reuse Call	Record Exists	Idle	Hit the consistency policy and directly return the historical virtual number.
Concurrency Conflict	Record Exists	In-Use	Trigger the Fallback mechanism, discard the old mapping, re-allocate a new number from the number pool and overwrite the mapping.

4.1.3 Control Plane Finite State Machine Design

As previously mentioned, to efficiently handle interactions between the control plane and the media plane—as well as API operations within the control plane—we opted to develop a Finite State Machine (Call State Machine). This state machine features rigorous abstraction and incorporates principles of defensive programming to manage the lifecycle of dual-callback calls. This model effectively prevents "dirty write" issues in the database caused by network latency, abnormal disconnections, or duplicate events. The state machine defines the following primary states and unidirectional transition rules:

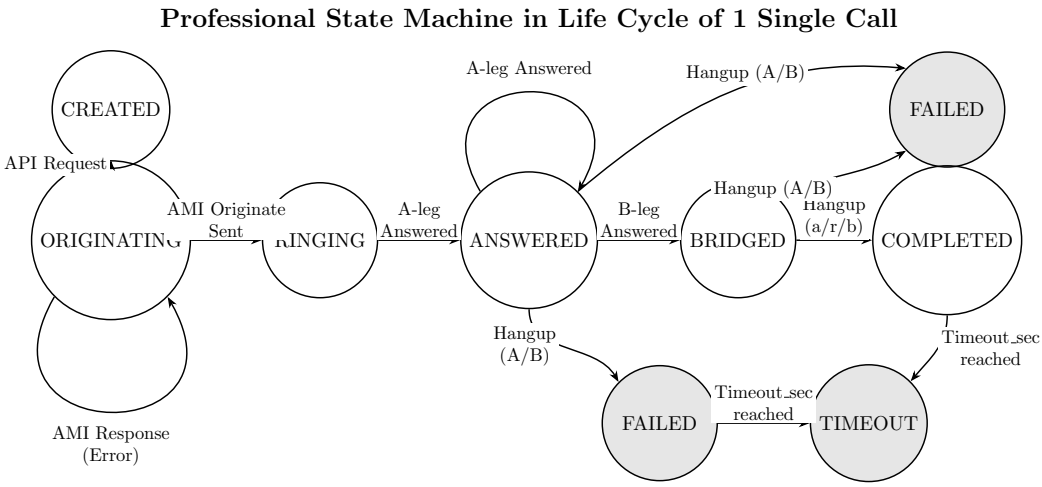

Figure 4.4: Professional State Machine in Life Cycle of 1 Single Call

- **Creation and Initialization Phase (CREATED → ORIGINATING):** When the business API completes number selection and persists the call metadata to SQLite, the session is in the initial CREATED state. Subsequently,

once the Node.js process assembles and issues the AMI commands, the system immediately transitions it to the ORIGINATING state. In this state, the system is waiting for the underlying protocol stack to return the initial transmission result.

- **Signaling Interaction and One-Sided Answer (RINGING → ANSWERED):** If the command is issued without errors, Leg A (the calling terminal) begins receiving the INVITE signal and rings, transitioning the state machine to RINGING. Once either party (Leg A or Leg B) clicks the answer button, Asterisk reports the state via a UserEvent, and the session state escalates to the ANSWERED state. This phase indicates that the system is actively working to bridge the remaining leg of the connection.

- **Full-Duplex Call Phase (BRIDGED):** When both the caller and the callee have answered, and Asterisk's internal Bridge engine has successfully completed the transcoding and port mixing mapping for the two RTP audio streams, the session immediately enters the BRIDGED state. A record in this state signifies the official beginning of a valid, billable call.

- **Termination and Exception Convergence States (COMPLETED / FAILED / TIMEOUT):** At the end of the call, the system branches into three different termination states depending on the reason for ending. If either the caller or callee actively hangs up, causing the destruction of Leg A/B, the state smoothly transitions to COMPLETED. If the Asterisk engine goes offline during the ORIGINATING phase, or if the AMI returns an Error response, it drops directly into the FAILED state. Additionally, the system features a built-in watchdog timer at the control layer; if the session continues ringing without bridging beyond the global parameter `timeout_sec`, the timer will forcibly interrupt the call and converge the state to TIMEOUT. All termination states are designed as sink states, strictly prohibiting any secondary backward transitions, thereby guaranteeing absolute consistency in billing data.

4.2 System Front-end Design

The front-end operations console adopts a lightweight, zero-dependency design philosophy. Given the stringent requirements for load speed, resource efficiency, and real-time responsiveness inherent to operations dashboards, the system eschews heavyweight frameworks—such as React or Vue—and is developed entirely using native HTML5, CSS3, and Vanilla JavaScript (ES6).

4.2.1 Front-end UI Layout Design

The page skeleton is constructed upon HTML5 semantic tags, utilizing elements such as ‘¡header¿‘, ‘¡main¿‘, ‘¡section¿‘, and ‘¡article¿‘ to clearly delineate the page’s logical structure. To facilitate complex data dashboard layouts, the front-end extensively employs CSS Grid and Flexbox. On mobile devices and small screens, the dashboard defaults to a single-column fluid layout; however, when the viewport width exceeds 1300px, the system triggers desktop-specific layout rules via media queries (‘@media (min-width: 1300px)‘). By leveraging the ‘grid-template-areas‘ property, the nine major functional modules—comprising Authentication, Overview, Alerts, Load, Services, Logs, Audit, Statistics, and Call Records—are seamlessly assembled into a dual-column asynchronous console panel with a 1.05:1.25 aspect ratio. This design ensures exceptional readability across devices of varying resolutions.

When viewed on a standard desktop browser aspect ratio, the upper section displays four distinct Grid Tiles. Located in the top-left corner is the authentication module. This module enables users to input necessary credentials—such as an API Key or User ID—as well as environment configuration settings. Upon receiving this information, the front-end securely stores it within the browser’s local cache (e.g., ‘localStorage‘ or in-memory variables); subsequently, for every outgoing Fetch request directed to the back-end, this information is automatically appended to the ‘Authorization‘ field within the HTTP headers, thereby ensuring the security of all API calls.

The front-end interface serves not merely as a display dashboard for data visualization, but also possesses the capability to directly control back-end processes—such as Asterisk and core API services. The Service Status section lists the system’s core

Pic/3.6.png

Figure 4.5: Server Front-end Snapshot 1

services (e.g., ‘asterisk.service‘ and ‘privacy-calling-api.service‘). In addition to displaying their current status (e.g., ‘active:running‘), Process IDs (PIDs), and startup timestamps, this section provides direct shortcut buttons for initiating ”start,” ”stop,” and ”restart” actions, thereby significantly streamlining the process management workflow. The Log section features native support for hierarchical search and retrieval. It facilitates rapid filtering based on service source (e.g., ‘asterisk‘, ‘api‘, ‘asterisk-full‘) and log level (e.g., ‘debug‘, ‘info‘, ‘warning‘, ‘error‘). The interface includes a front-end keyword filter box, along with features such as ”Pause Tail” (to facilitate the review of scrolling logs), manual refresh, and ”Export JSON” for offline analysis purposes. Log content is presented in a structured tabular format—organized by timestamp, service, level, and message body—to facilitate the quick retrieval of specific information.

Pic/4.1.png

Figure 4.6: Server Front-end Snapshot 2

The bottom section focuses primarily on business-level data metrics and the logging of administrative actions. The Operation Audit area records the administrator’s activity trail on the dashboard in the form of a chronological timeline (with

data persisted to SQLite). Call Statistics provides a high-level overview of business performance via a set of data cards. These metrics include the total number of calls, current active calls, call volume and failure rates over the past 24 hours, as well as the system's current quotas for enabled users and virtual numbers (e.g., 3/3). The Recent Calls section presents detailed call logs in a tabular format. This includes the client-generated Call ID, current call status (e.g., "completed," "failed"), the Caller and Callee numbers, the assigned intermediary virtual number, and the record's creation timestamp—facilitating communication tracking for specific transactions by customer support or technical personnel.

Pic/4.2.png

Figure 4.7: Server Front-end Snapshot 3

4.2.2 Front-end UI Logic Design

The core of the frontend business logic revolves around a global 'state' object. This object centrally manages all critical data within the system, including the API key, an array of logs, historical monitoring metrics, and filtering criteria (such as 'selectedLogServices'). In terms of interaction logic, the system adheres to the principle of data-driven views. When a user performs an action—such as modifying the log filtering level—JavaScript modifies only the internal data within the 'state' object; it then invokes the corresponding rendering function (e.g., 'renderLogs()') to reconstruct and replace the interface elements using DOM APIs.

To ensure the security of API calls, the frontend has extensively encapsulated the native Fetch API (establishing a unified 'request' method). This method automatically checks for and retrieves the user's cached 'privacy_calling_api_key' from 'localStorage', injecting an 'x-api-key' into the HTTP headers of every outgoing request. The 'request' function incorporates a built-in Promise 'catch' mechanism; should the

Pic/4.pdf

Figure 4.8: Server Front-end Logic Design

backend return a non-OK HTTP status code, the current call stack is automatically interrupted. The error message—extracted from ‘payload.error.message‘—is then relayed to the administrator via a low-level Toast notification component as a non-blocking alert.When a user enters input into the search box, the system does not immediately initiate a backend query; instead, it employs the ‘getFilteredLogsByKeyword‘ function to filter the ‘state.logs‘ array—which is cached in memory—using higher-order functions (specifically, a combination of ‘filter‘ and ‘includes‘). This ”space-for-time” strategy delivers a purely frontend log retrieval experience with millisecond-level responsiveness. The system also supports a one-click log export feature. Rather than requesting the server to deliver a file, the frontend directly extracts the currently filtered log array; it then utilizes the HTML5 Blob API to dynamically construct a Binary Large Object (Blob) of the ‘application/json‘ type. By generating a temporary URL via ‘URL.createObjectURL‘ and simulating a click on an ‘¡a¿‘ tag, the system enables the direct local generation and download of a ‘.json‘ audit report. This design not only safeguards data security but also completely eliminates any backend I/O overhead.

4.3 Server Back-end Design

4.3.1 Back-end Function Design

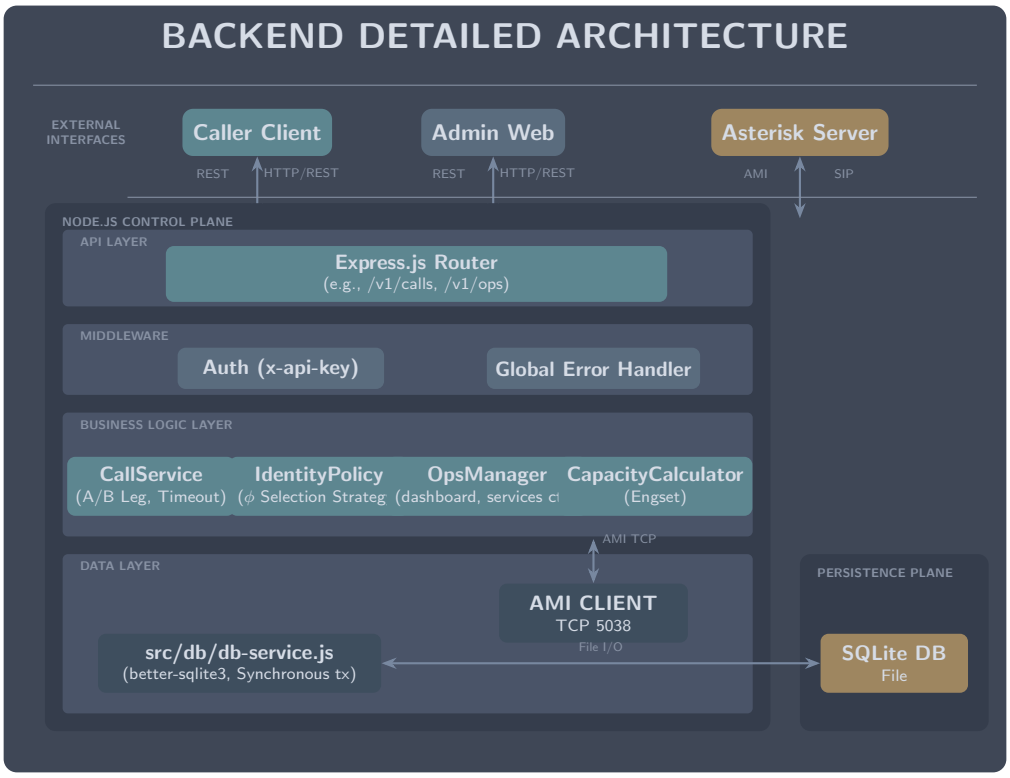

Figure 4.9: Backend Detailed Architecture

The system's fundamental communication capabilities are built upon a bidirectional callback mechanism. When a caller initiates a call request via the API, the backend does not route the call directly from the caller to the callee; instead, it actively serves as an intermediary. The implementation of this functionality proceeds in two distinct steps: Step 1 (A-Leg)—the server instructs the Asterisk engine to initiate a call to the SIP endpoint registered by the caller. This ensures that the caller need only answer the incoming call to establish communication, thereby incurring no personal telephony charges. Step 2 (B-Leg)—once the caller answers and enters the system's designated "_bridge," the system engine immediately retrieves a pre-selected virtual number; by dynamically rewriting the 'CALLERID(num)' field within the SIP packets, it then initiates a call to the actual callee, presenting itself as that virtual number. Consequently, when the callee's device rings, only this virtual number appears on the screen, thereby completely severing—at the physical

signaling level—any potential pathway for the caller's true identity to be exposed.

Furthermore, to ensure a highly time-sensitive and secure privacy loop, the backend incorporates a rigorous timeout control flow within the interactions between the ‘createApp‘ and ‘CallService‘ components. Call requests may optionally include a ‘timeout_sec‘ parameter; the system clamps this value between a default setting (e.g., 30 seconds) and a configured absolute maximum limit. This timeout parameter serves a dual purpose: it not only limits the time the API waits for a response but is also injected as a variable into the underlying Asterisk ‘Originate‘ action. Should this threshold be exceeded—meaning neither party has answered—the system automatically dispatches a ‘Cancel‘ command to terminate the session and release the virtual number back into the pool. Concurrently, the system features an automatic routing resolution capability designed to handle complex destination numbers: it prioritizes matching against configured external Trunks (trunk routes) for dialing actual mobile phone numbers; conversely, if operating within an internal testing environment, it automatically resolves and maps the destination to a local SIP extension, thereby ensuring seamless compatibility for dual-track communication across both external and internal networks.

4.3.2 Database Design

This system employs a minimalist yet highly efficient, lightweight relational data persistence solution, utilizing SQLite as a substitute for the resource-intensive combination of MySQL and Redis. This approach not only significantly reduces runtime memory consumption but also eliminates issues regarding delayed signaling state updates caused by network latency. The entire database design is firmly centered around the core principles of Entity-Relationship modeling.

Within the system's underlying data model, Users and Virtual Numbers constitute the two most fundamental entities. The User table securely stores the caller's actual international standard telephone number (in E.164 format) along with their terminal device information; conversely, the Virtual Number table is responsible for managing the availability status of each number within the system's number pool. When communication activity occurs, this foundational data converges into the system's core transaction table—the Master Call Record table. For every call, a

primary record is generated here, providing a clear log of the calling user's identity, the specific virtual number allocated to them by the system for that instance, the ultimate destination number, and the overall connection status of the current call.

To address the complex routing and tracking requirements inherent in modern communication systems, this design also incorporates two auxiliary tables: Call Branches and Mapping Relationships. In actual communication scenarios, a complete call often comprises multiple distinct network connection segments; the Call Branch table enables granular, independent monitoring of the type and status of each individual segment. Concurrently, the Mapping Relationship table is dedicated to handling specific business binding logic—such as establishing temporary or long-term associations between a destination number and a specific virtual number—thereby ensuring the accuracy of communication routing.

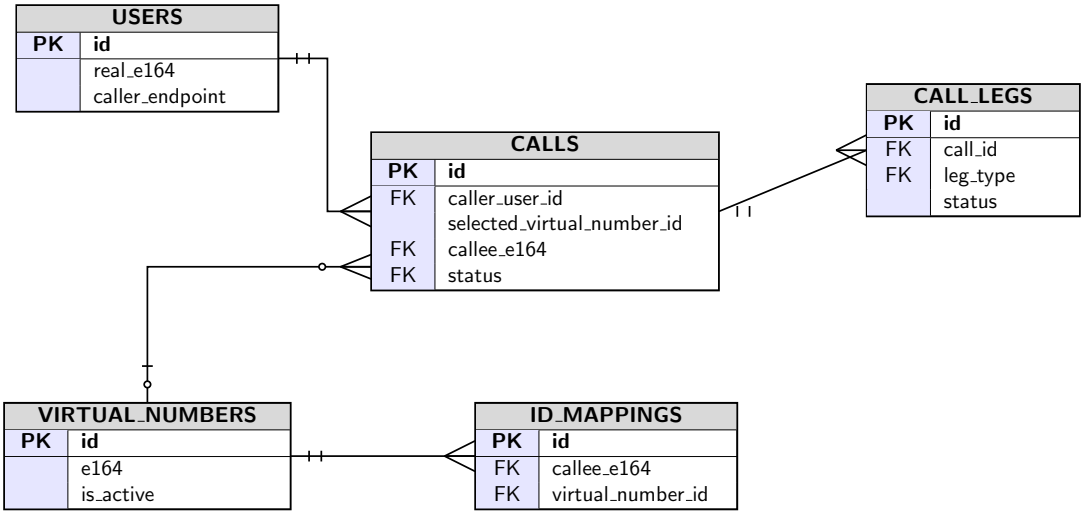

Figure 4.10: Database Entity-Relationship Diagram

4.3.3 System Deployment

Regarding system architecture evolution, in order to maximize the throughput of the hardware network stack and circumvent complex Network Address Translation (NAT) issues, the current deployment design explicitly eschews Docker containerization in favor of a purely native Linux deployment scheme—utilizing either physical machines or virtual machines. This design enables Asterisk to interface directly with the host system's kernel network modules, thereby ensuring high-quality transmission of real-time audio streams (RTP).

The designated deployment environment is a Debian-based Linux distribution (e.g., Ubuntu). To simplify the installation process, the project incorporates a robust, automated Bash script (‘sudo ./scripts/install-system.sh‘). This script handles the one-step download, compilation, and installation of specific versions of Asterisk core components and their critical dependencies (such as the PJSIP protocol stack), as well as the configuration of the Node.js and npm runtime environments. By implementing Infrastructure as Code, the time required to deploy the system from scratch has been significantly reduced.

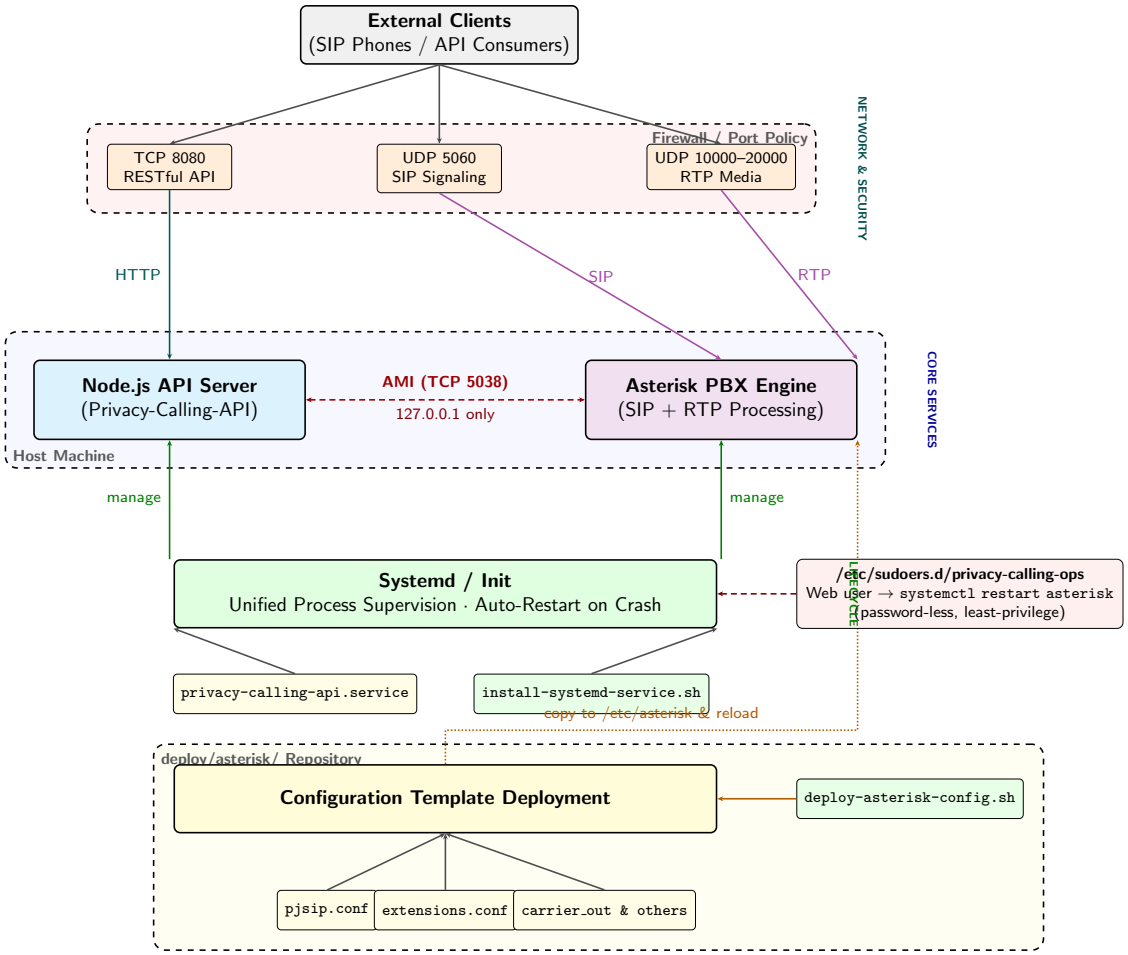

Figure 4.11: System Deployment Flow

Chapter 5

RESULTS AND EVALUATION

Chapter 6

CONCLUSIONS AND FUTURE WORK

6.1 Conclusions

Summary of the main take-away points from the project.

6.2 Future Work

Possibilities for further extensions of the predictive tools.

REFERENCES

Appendix A

APPENDIX TITLE